Machine Learning-Based Remaining Useful Life Estimation and Maintenance Alerting for Aircraft Turbofan Engines

TE7483 Applications and Use Cases of Machine Learning ? AIML Dept., B.Tech Sem 5 ? Final ESE case-study project

Authors: [Team / names to fill] ? Date: October 2026

Contents

(In Word: right-click -> Update Field to populate.)

Abstract

We predict the remaining useful life (RUL) of aircraft turbofan engines from multivariate sensor histories on the NASA C-MAPSS benchmark and convert RUL into an actionable maintenance-alert policy (safe / warning / critical; fails-within-30-days analogue in cycles). On FD001 the CV-selected model (hgb) reaches a test RMSE of 12.35 cycles on capped RUL (MAE 9.02, PHM score 240.7) with a grouped 5-fold CV RMSE of 11.61 ? 0.43. The same leakage-free pipeline is repeated on FD002-FD004 (multi-condition, two fault modes) with operating-condition normalisation. Alert thresholds are tuned on out-of-fold predictions at realistic truncation points and frozen before touching the test set; the deployed policy attains 1.00 fails-within-30 recall on all four test subsets with median warning lead times of 50-53 cycles. All numbers are reproduced end to end by `python run_pipeline.py --fresh`; every table and figure below is generated from files in results/.

1. Introduction and problem refinement

The vague brief - "predict engine failure" - is refined into two measurable tasks. (i) Regression: estimate remaining useful cycles from the sensor history available up to the current cycle, evaluated per engine at its last observed cycle against the official RUL labels, primarily on capped RUL (Section 3). (ii) Alerting: map RUL to safe (RUL > 50), warning (20-50), critical (< 20) and to a binary fails-within-30 flag, evaluated with precision, recall, F1 and confusion matrices with emphasis on critical recall. Stakeholders are airline maintenance planners (work packaging), MRO engineers (inspection decisions) and fleet safety managers (risk). Real-world value comes from replacing fixed-interval preventive maintenance with condition-based predictive maintenance: fewer unscheduled removals, fewer wasted healthy-life inspections. Assumptions: C-MAPSS trajectories are simulated run-to-failure / truncated histories with one snapshot per cycle and no in-trajectory repair; operating settings are observed; costs used in Section 6 are assumed (10:1), not airline data. Success criteria, fixed before seeing v2 results: (C1) FD001 test RMSE < 20 on capped and uncapped RUL; (C2) critical recall >= 0.85 on all subsets; (C3) fails-within-30 recall >= 0.95 on all subsets; (C4) median warning lead time > 10 cycles. Scored in Section 7.

2. Related work

Statistical and similarity methods. Wang, Yu, Siegel and Lee [2] introduced trajectory-similarity RUL prediction, the winning family of the PHM08 data challenge: a test trajectory is matched against a library of run-to-failure histories. Zio and Di Maio [6] predict RUL by fuzzy similarity to reference failure trajectories with online updating. Peel [4] won the IEEE Gold in the PHM08 challenge by combining neural network ensemble members with a Kalman filter. Our work differs in using supervised discriminative models with engineered causal features rather than trajectory matching, but keeps the PHM08 asymmetric scoring function as the cost-aware regression metric.

Recurrent and convolutional models. Heimes [3] applied recurrent networks to C-MAPSS RUL at PHM08. Zheng et al. [8] showed LSTM sequence models exploiting multi-condition sensor histories (IEEE ICPHM 2017). Babu, Zhao and Li [7] proposed a deep CNN regressor for RUL (DASFAA 2016), and Li, Ding and Sun [10] used deep convolutional networks with dropout for prognostics (2018). Zhang et al. [9] evolved an ensemble of deep belief networks under accuracy-diversity objectives (MODBNE). Peng et al. [5] used modified echo-state networks for RUL. Our deep baselines (1D-CNN, LSTM on length-30 causal windows) follow this line but are held to the same grouped validation protocol as the classical models - and, as Section 6 shows, do not beat gradient boosting on this feature set, a result we report rather than hide.

Uncertainty and explanation. We use split/Mondrian conformal prediction in the framework of Vovk, Gammerman and Shafer [12] for prediction intervals, and SHAP values [11] alongside permutation importance for tree-model explanation. The dataset and challenge protocol come from Saxena et al. [1]. What is novel here is not a new architecture but a leakage-free, fully reproduced comparison with a defensible alert policy: the normalise-before-smoothing fix (Section 4) was found by inspecting which features the model relied on, and is documented as an ablation.

3. Dataset

Source. NASA C-MAPSS turbofan degradation simulation [1] (readme and PDF in data/raw/). Train trajectories run to failure; test trajectories are truncated before failure with true RUL at the last observed cycle given in RUL_FD00x.txt (asserted equal to the number of test units). Files are whitespace-separated with no header and exactly 26 columns (a 27th all-NaN column from trailing spaces is dropped by the loader, which raises otherwise). Engines start with different unknown initial wear (normal, not a fault); sensor noise is present. Subsets: FD001 (1 condition, HPC fault), FD002 (6 conditions, HPC), FD003 (1 condition, HPC+Fan), FD004 (6 conditions, HPC+Fan).

Table 1. Verified dataset sizes and lifetimes (loader-asserted; note FD004 has 249 train / 248 test units, the readme's 248/249 is off by one).

subset	train rows	test rows	train units	test units	life min/max/mean	test last-RUL mean
FD001	20631	13096	100	100	128/362/206	75.500
FD002	53759	33991	260	259	128/378/207	81.200
FD003	24720	16596	100	100	145/525/247	75.300
FD004	61249	41214	249	248	128/543/246	86.600

Interpretation. FD002/FD004 are 2-3x larger with shorter, more variable lives; the multi-condition and dual-fault subsets are the generalisation test.

Table 2. The 26 columns (no header in files).

columns	meaning	dtype
unit	engine id	int
cycle	cycle number	int
op1-op3	operating settings	float x3
s1	fan inlet temp	float
s2	LPC outlet temp	float
s3	HPC outlet temp	float
s4	LPT outlet temp	float
s5	fan inlet pressure	float
s6	bypass-duct pressure	float
s7	HPC outlet pressure	float
s8	physical fan speed	float
s9	physical core speed	float
s10	engine pressure ratio	float
s11	HPC outlet static pressure	float
s12	fuel flow / phi	float
s13	corrected fan speed	float
s14	corrected core speed	float
s15	bypass ratio	float
s16	burner fuel-air ratio	float
s17	bleed enthalpy	float
s18	demanded fan speed	float
s19	demanded corrected fan speed	float
s20	HPT coolant bleed	float
s21	LPT coolant bleed	float

Table 3. Test-set class counts over all rows (safe dominates everywhere; last-cycle counts differ - see confusion matrices).

fd	n_engines	safe	warning	critical	leq30	note
FD001	100	12210	779	107	332	counts over ALL test rows; last-cycle counts differ
FD002	259	31405	2135	451	1087	counts over ALL test rows; last-cycle counts differ
FD003	100	15817	671	108	291	counts over ALL test rows; last-cycle counts differ
FD004	248	38964	1916	334	864	counts over ALL test rows; last-cycle counts differ

Interpretation. Class imbalance is structural: most cycles of most engines are healthy. Accuracy is therefore never used for selection; recall on critical / fails-within-30 is the safety metric.

Table 4. Sensors kept/dropped per subset (variance on normalised train data; s1/s5/s18/s19 drop on FD002/FD004 because they are constant within each condition).

subset	kept	dropped
FD001	15	s1, s19, s18, s5, s16, s10
FD002	17	s18, s5, s19, s1
FD003	16	s1, s19, s18, s16, s5
FD004	17	s19, s18, s5, s1

Targets. Train RUL = max cycle of unit ? cycle; test RUL for every row = RUL_last(unit) + (last_cycle ? cycle). Regression is trained on piece-wise linear RUL capped at 125 (degradation is not observable early in life; flat sensors would otherwise force the model to fit noise) and evaluated primarily on capped RUL - the literature convention - with uncapped metrics kept as secondary. Because all class thresholds (50/20/30) lie below the cap, the cap cannot distort the labels. No missing values, no duplicate rows, dtypes validated in src/data.py. Validation: GroupKFold (5-fold) and GroupShuffleSplit strictly by engine; scalers, KMeans, selectors fit on training folds only; test features use past data of that engine only (causal rolling windows; short histories edge-padded with the first row - stated rule). Two leakage bugs were found and fixed during this project and are regression-tested: (i) a cycle/max_cycle feature that encoded failure time (removed); (ii) smoothing raw sensors before condition normalisation, which blended operating modes (fixed order: normalise -> drop -> smooth -> rolling; the bug is kept as a documented ablation in Section 6).

4. Methodology

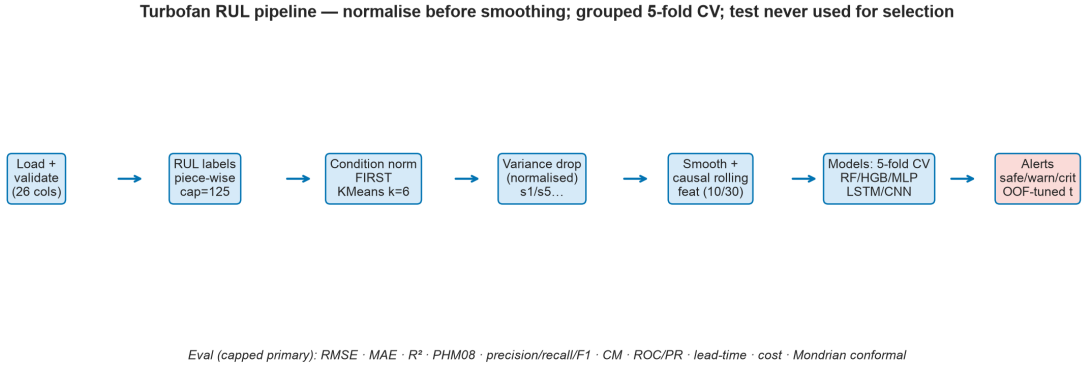


Figure 1. End-to-end pipeline. Fits happen on train only; model selection uses grouped CV; thresholds are frozen on out-of-fold predictions before the test set is touched.

Interpretation. The diagram order matches the code exactly. The key decision - normalise before smoothing - is what makes the multi-condition subsets tractable (Section 6, smooth-order ablation).

Preprocessing. (i) Operating-condition normalisation: KMeans (k=6, elbow + silhouette evidence in Figures for FD002/FD004) on standardised op settings, then z-score each sensor within its cluster for FD002/FD004; global standardisation for FD001/FD003. Within-condition std at float-noise level (~1e-13, e.g. s1/s5/s19) is floored so these sensors normalise to exactly 0 and drop out, instead of having their numerical dust amplified to ?1. (ii) Causal exponential smoothing (?=0.3) per unit on normalised sensors. (iii) Causal rolling mean/std/slope over windows [10, 30] plus raw cycle and op settings; train-fit StandardScaler. (iv) Sliding windows (length 30, ablation 20/30/50) of normalised sensors for deep models.

Models and rationale. Ridge (linear reference) and mean-RUL (floor); Random Forest and HistGradientBoosting (nonlinear tabular workhorses; XGBoost/LightGBM wheels require system libomp, absent on this machine - HGB is the honest boosting representative); MLP (dense nonlinear baseline); 1D-CNN and 1-layer LSTM on causal windows (sequence inductive bias), early stopping on by-engine holdout, seeds 42-44. Classification is done both directly (logreg/RF/HGB with class weights) and as RUL-regress-then-threshold, compared head to head. Tuning: RandomizedSearch over HGB (5-fold grouped CV). Imbalance: class_weight balanced (ablated off). Thresholds: out-of-fold RUL predictions of the deployed model at random truncation points per engine; pick the smallest-cost threshold subject to OOF recall >= 0.95, else the cost-optimal threshold with target_met=false (costs assumed 10:1, sensitivity in Section 6). Uncertainty: Mondrian split-conformal intervals binned on predicted RUL.

5. Implementation

Python 3.12, scikit-learn 1.9.1, PyTorch 2.14.1 (CPU), matplotlib/seaborn; repository aucml_rul_project with src/ (data, preprocess, models, train, evaluate, explain, alerts, plots), config.yaml, Makefile and run_all.sh. Reproduce: `pip install -r requirements.txt && python run_pipeline.py --fresh` (subset wall-clock: FD001 17.6 min, FD002 37.6 min, FD003 24.2 min, FD004 51.5 min), then scripts/build_report.py, build_slides.py; `pytest tests/ -q` (12 tests); notebooks execute via nbconvert; `streamlit run app.py` demo (upload/choose an engine history -> predicted RUL, green/amber/red alert, trajectory). Seeds 42-44; pinned requirements for imported packages; all artefacts under results/, figures/, models/.

6. Results and comparative analysis

6.1 RUL regression (capped primary)

Table 5. Test RMSE on capped RUL @125, last cycle per engine (primary metric; lower is better).

model	FD001	FD002	FD003	FD004
cnn	14.570	16.510	17.650	18.510
hgb	12.350	12.870	13.170	13.170
hgb_tuned	12.240	12.830	12.880	12.970
lstm	13.750	14.880	13.160	14.770
mlp	13.500	14.580	14.680	13.400
rf	12.270	13.250	14.660	13.730
ridge	15.660	16.350	15.910	18.350

Interpretation. HGB is CV-selected on all four subsets. The normalise-first fix moves FD002 HGB 18.27->12.87 and FD004 HGB 23.87->13.17; single-condition subsets are flat (FD001 11.63->12.35 mixes the 100->300 estimator change - at fixed 100-iter capacity the fix is 12.37->12.04, i.e. neutral-to-positive). Deep models trail boosting by 1-4 RMSE points everywhere.

Table 6. Grouped-CV RMSE (capped, mean ? std; deep = by-engine holdout) beside capped test RMSE. Selection used the CV column only.

subset	model	CV RMSE	test RMSE cap	test PHM cap
FD001	ridge	15.81 ? 0.59	15.660	355.700
FD001	rf	12.65 ? 0.27	12.270	210.900
FD001	hgb	11.61 ? 0.43	12.350	240.700
FD001	mlp	14.62 ? 0.57	13.500	300.400
FD001	hgb_tuned	11.48 ? 0.47	12.240	225.100
FD001	lstm	12.98 ? 1.26	13.750	388.300
FD001	cnn	15.65 ? 1.10	14.570	370.100
FD002	ridge	16.73 ? 0.28	16.350	1215.000
FD002	rf	13.87 ? 0.34	13.250	862.100
FD002	hgb	13.19 ? 0.32	12.870	736.900
FD002	mlp	16.16 ? 0.32	14.580	1266.400
FD002	hgb_tuned	12.99 ? 0.30	12.830	762.500
FD002	lstm	16.57 ? 1.09	14.880	1143.600
FD002	cnn	21.97 ? 3.87	16.510	1319.900
FD003	ridge	15.31 ? 0.67	15.910	465.200
FD003	rf	10.56 ? 1.59	14.660	532.200
FD003	hgb	9.85 ? 1.47	13.170	313.300
FD003	mlp	11.97 ? 1.45	14.680	588.100
FD003	hgb_tuned	9.81 ? 1.50	12.880	311.400
FD003	lstm	12.01 ? 0.35	13.160	292.800
FD003	cnn	16.52 ? 1.59	17.650	846.800
FD004	ridge	17.55 ? 0.64	18.350	1602.400
FD004	rf	13.58 ? 0.99	13.730	839.100
FD004	hgb	13.06 ? 1.12	13.170	856.700
FD004	mlp	16.13 ? 1.03	13.400	924.900
FD004	hgb_tuned	12.96 ? 1.07	12.970	752.200
FD004	lstm	16.14 ? 0.62	14.770	1015.200
FD004	cnn	18.24 ? 0.83	18.510	1982.400

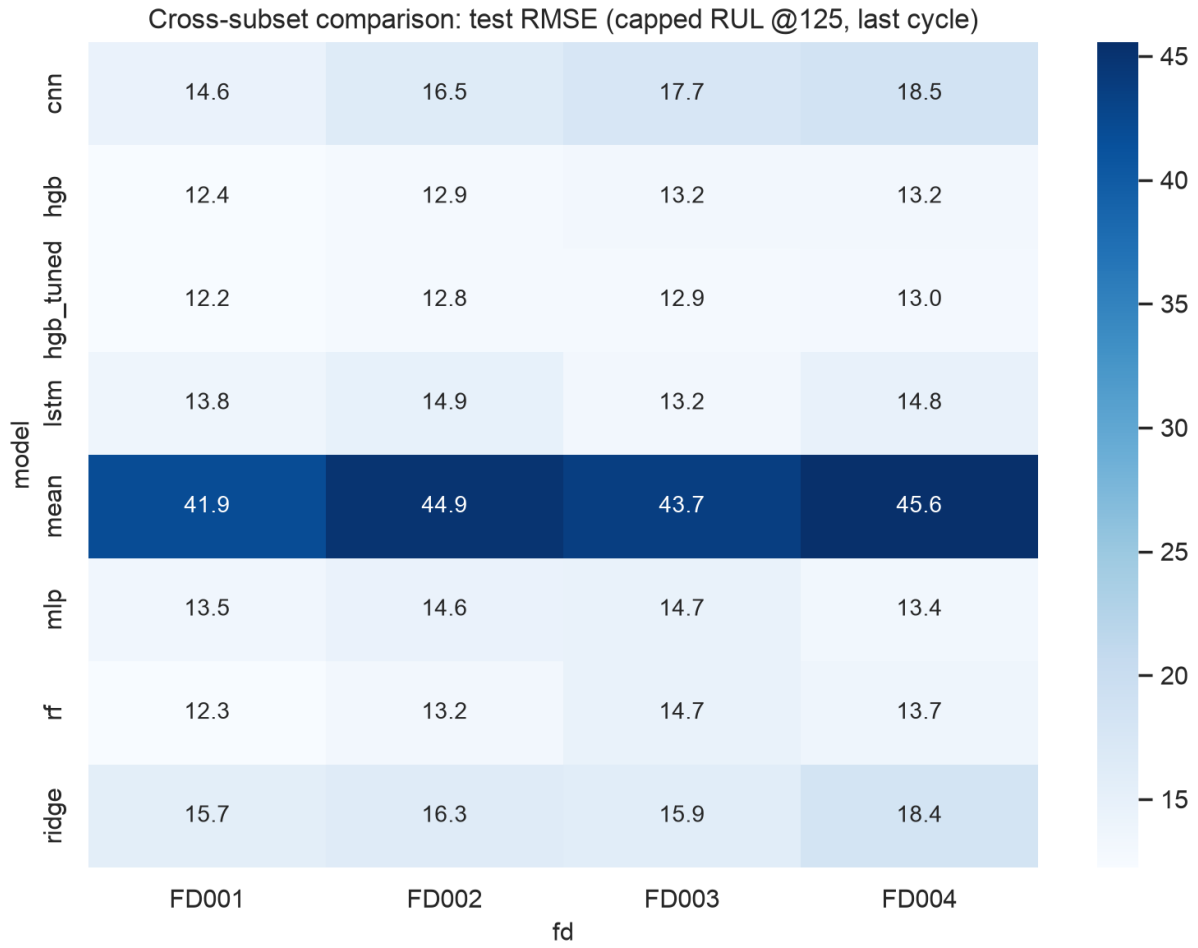


Figure 2. Cross-subset heatmap of capped test RMSE: multi-condition/fault subsets remain harder, but the gap closed from ~2x to ~7%.

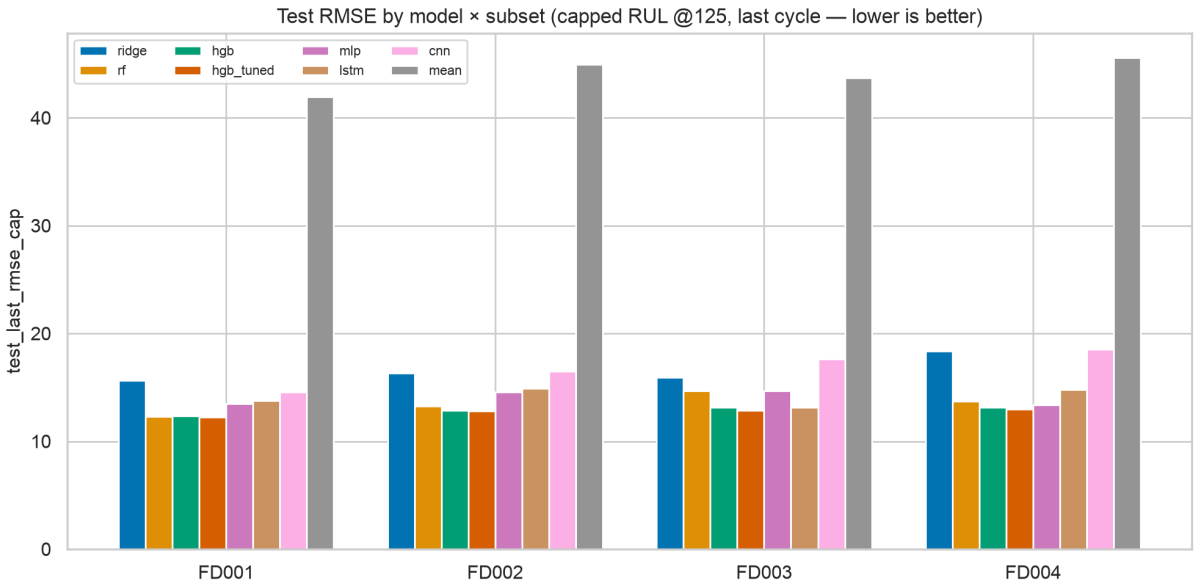


Figure 3. Grouped capped-RMSE bars: boosting leads on every subset; tuned HGB ? default HGB.

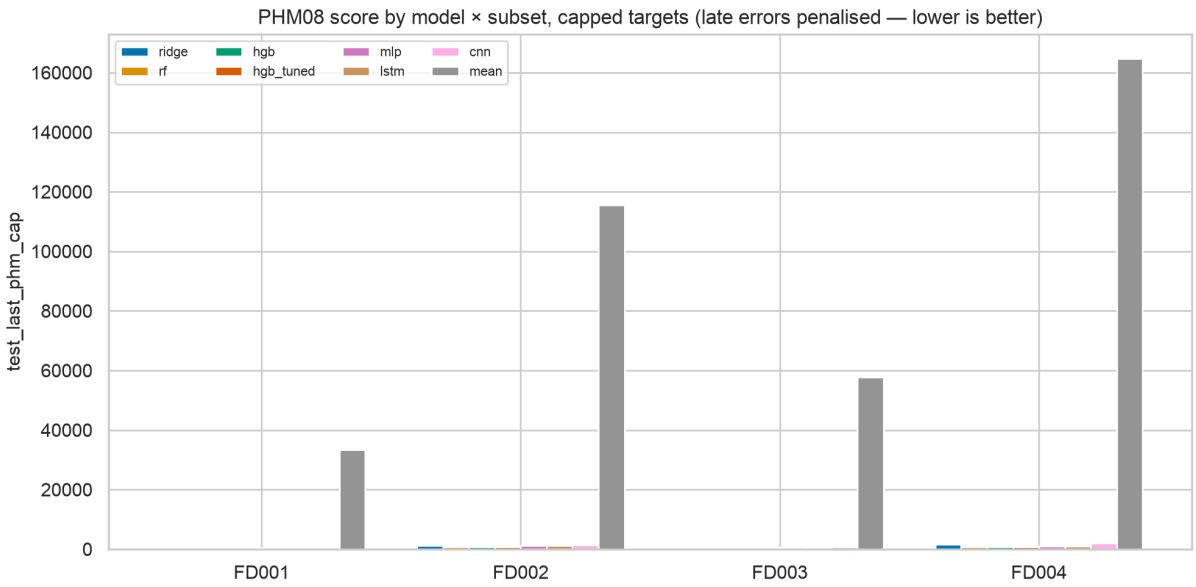


Figure 4. PHM08 scores on capped targets: late errors penalised asymmetrically; ranking mirrors RMSE.

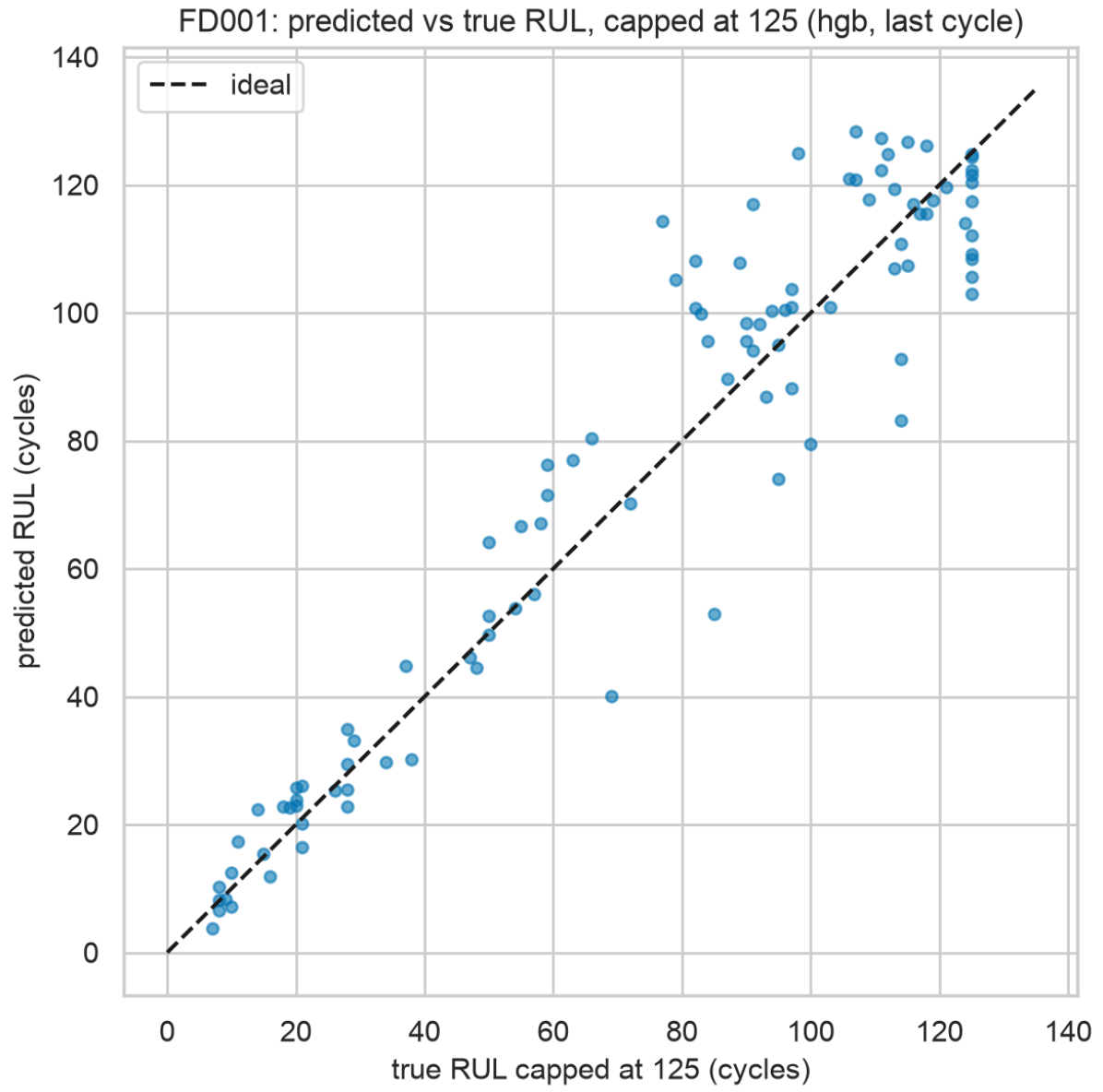


Figure 5. FD001 predicted vs capped-true RUL (hgb, last cycle).

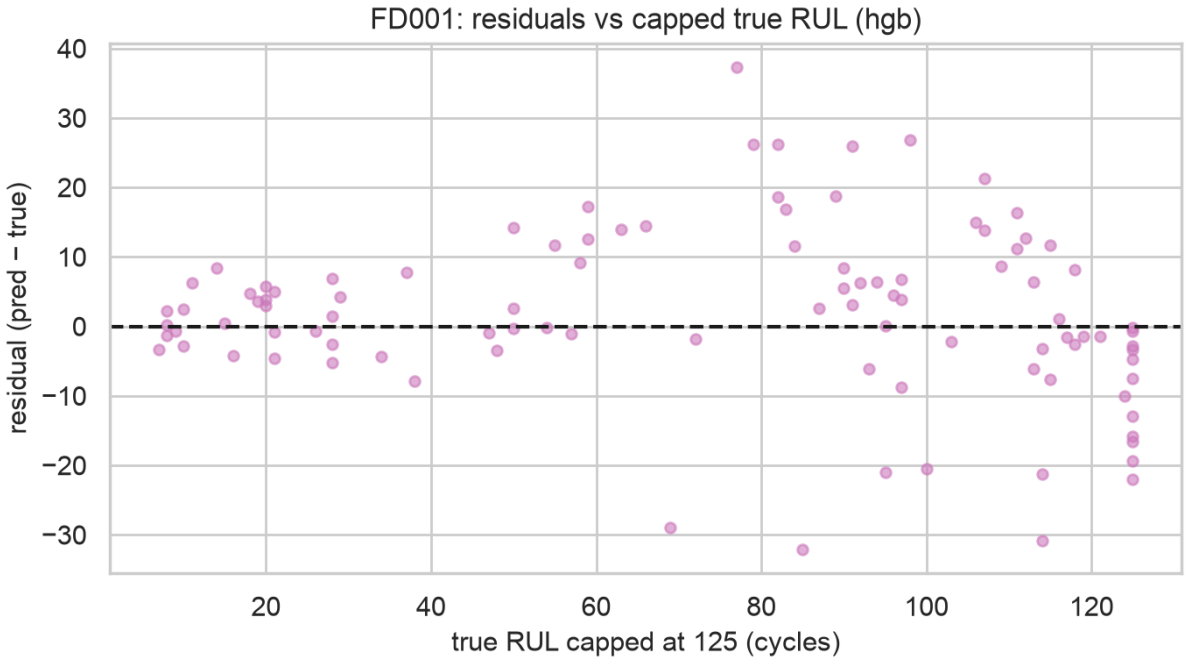


Figure 6. FD001 residuals vs capped-true RUL (hgb): errors grow far from failure, shrink near it.

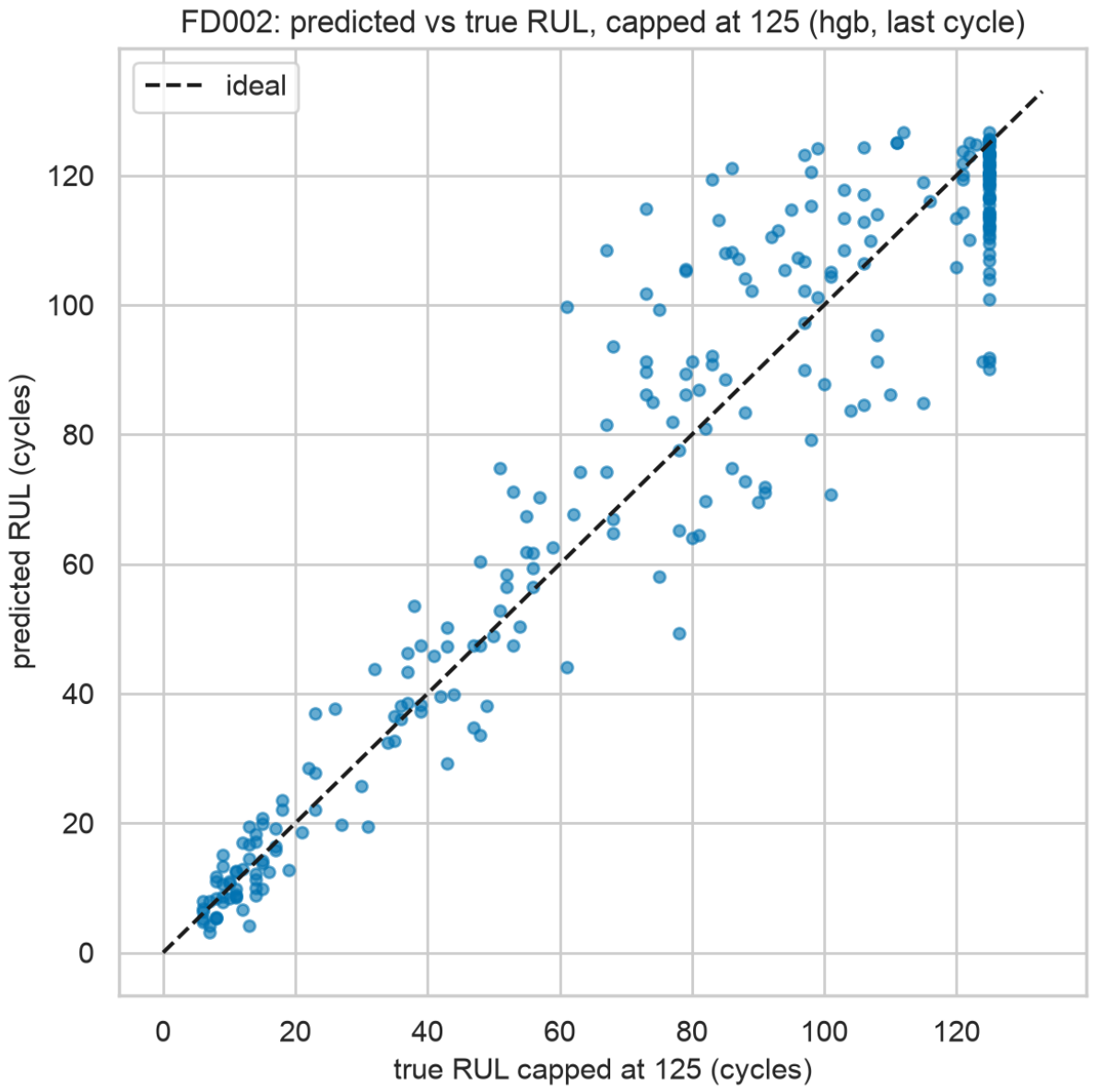


Figure 7. FD002 predicted vs capped-true RUL (hgb, last cycle).

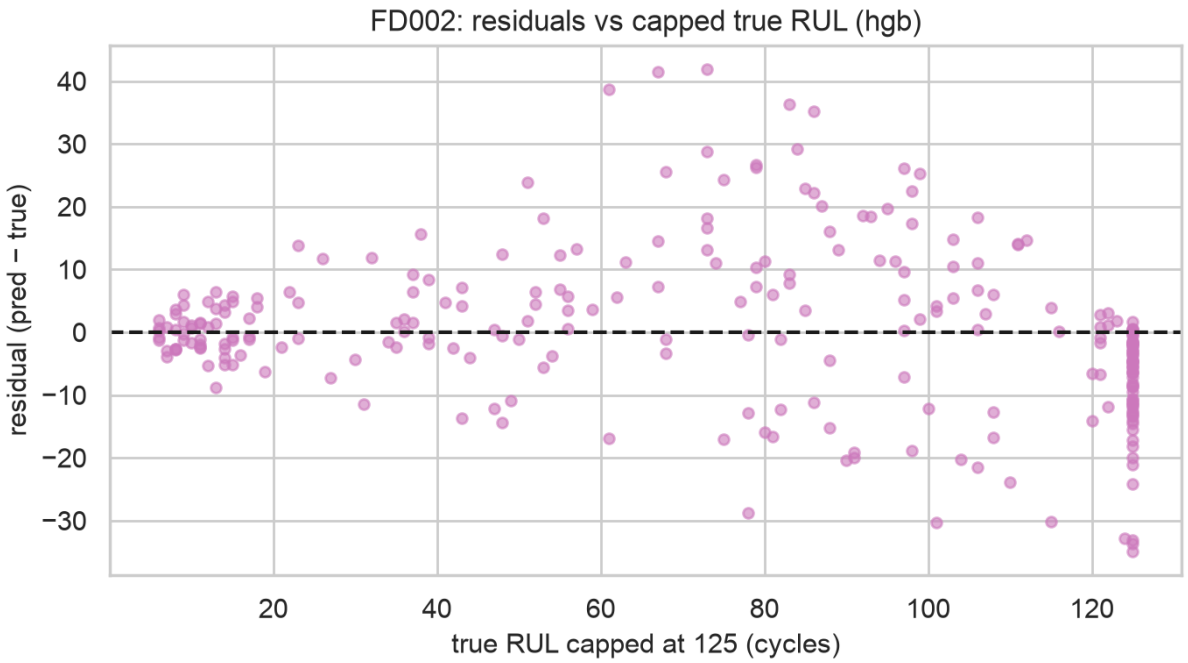


Figure 8. FD002 residuals vs capped-true RUL (hgb): errors grow far from failure, shrink near it.

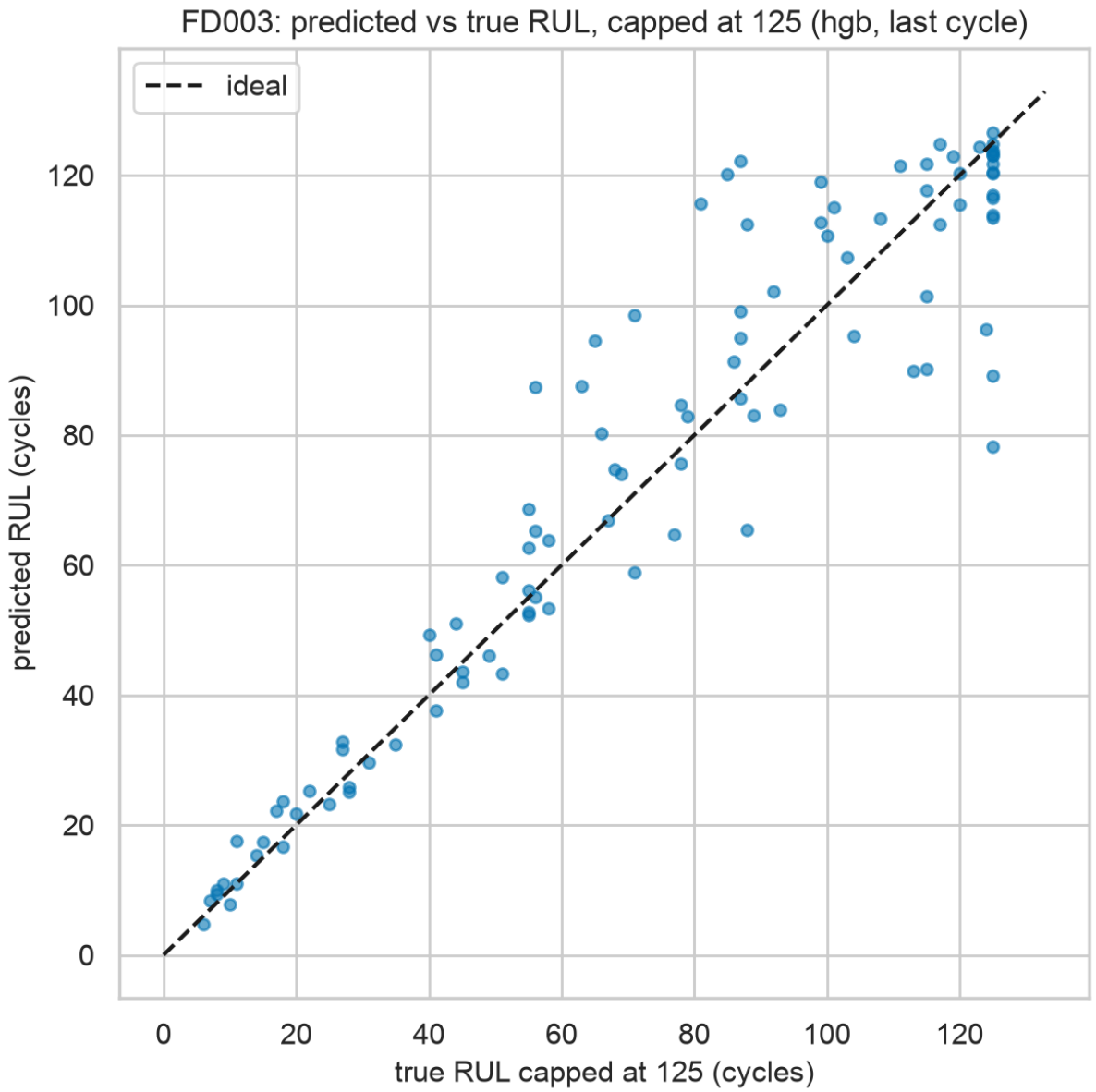


Figure 9. FD003 predicted vs capped-true RUL (hgb, last cycle).

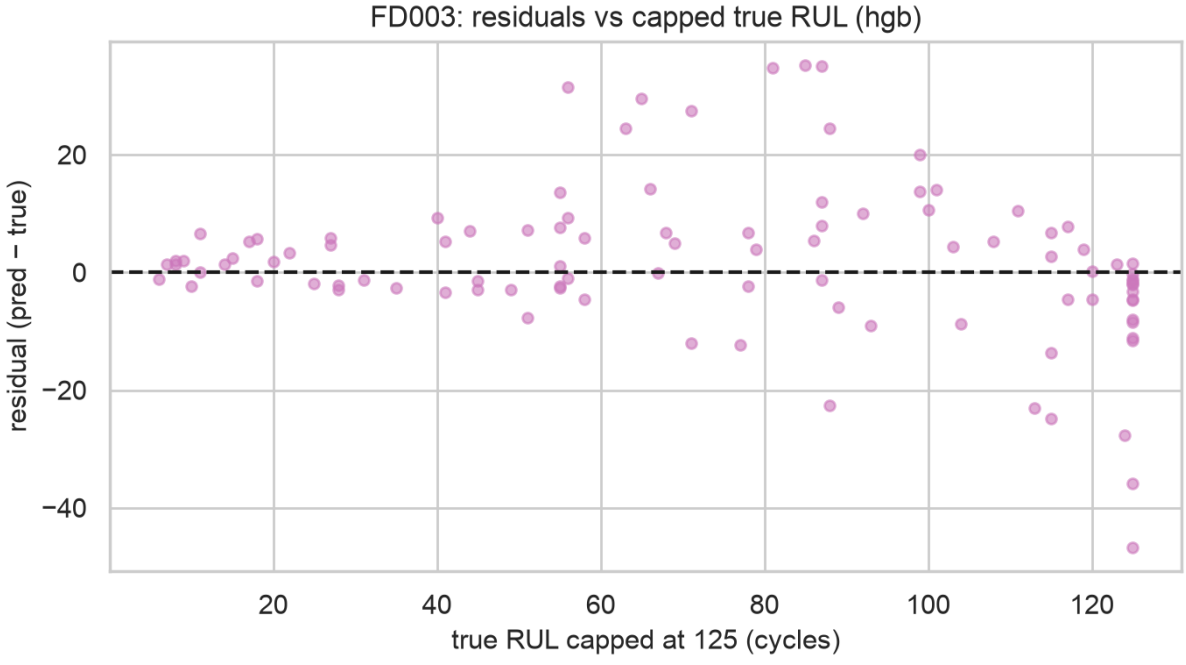


Figure 10. FD003 residuals vs capped-true RUL (hgb): errors grow far from failure, shrink near it.

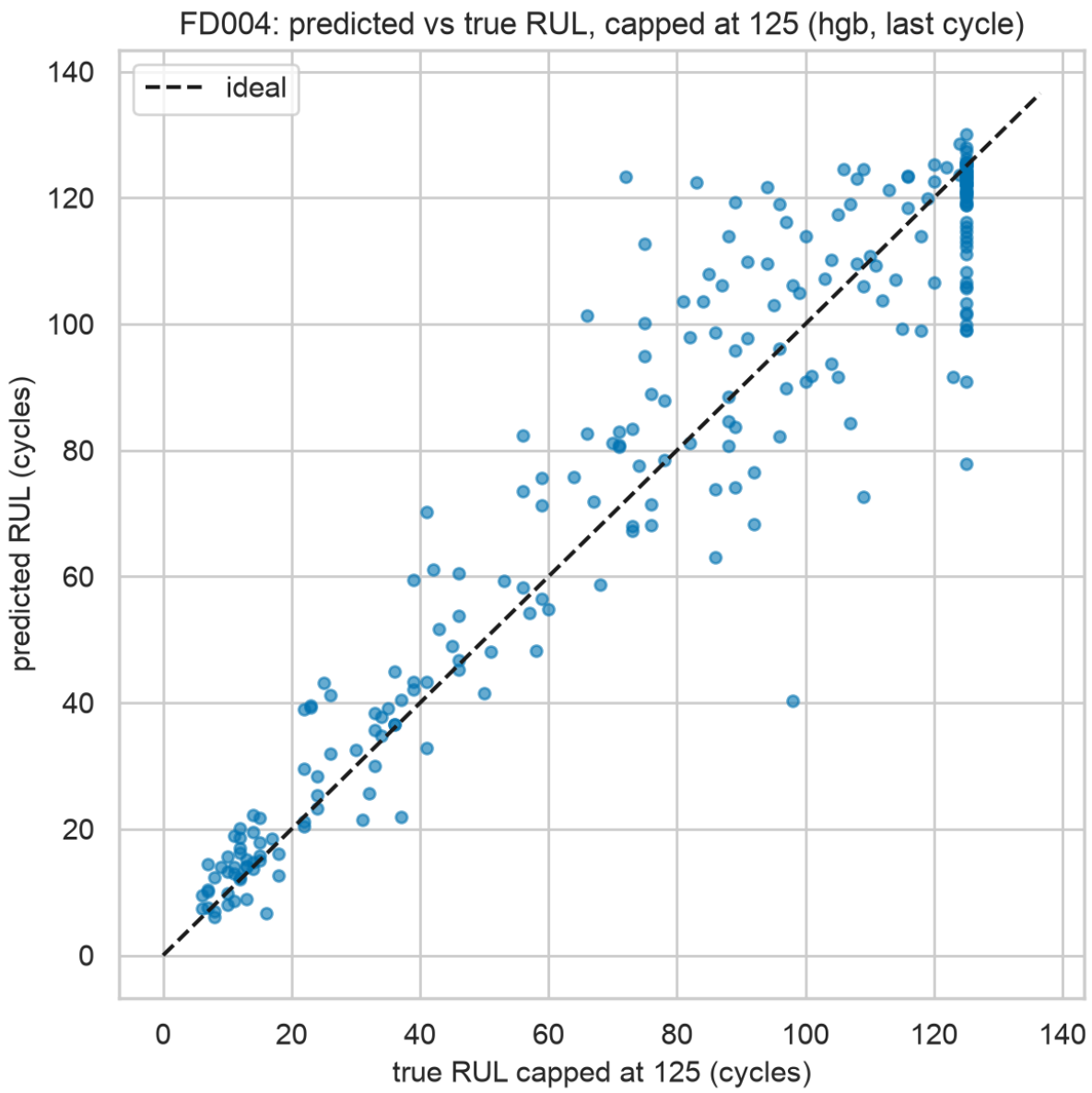


Figure 11. FD004 predicted vs capped-true RUL (hgb, last cycle).

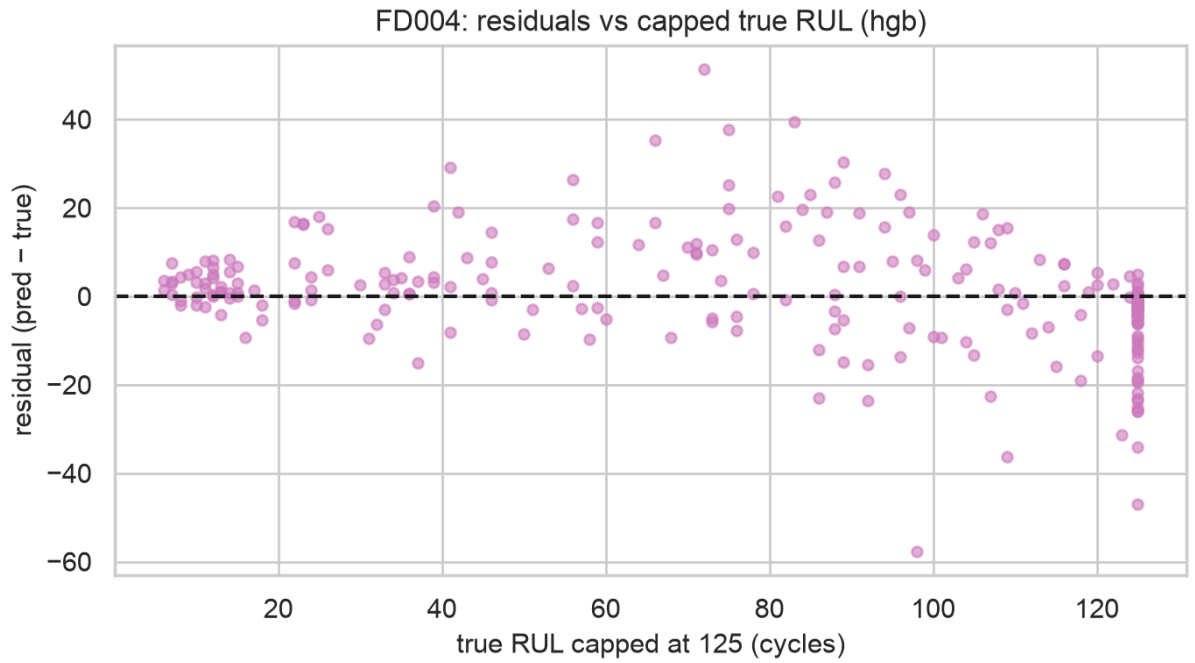


Figure 12. FD004 residuals vs capped-true RUL (hgb): errors grow far from failure, shrink near it.

Table 7. Per-RUL-bin errors (deployed model, capped last-cycle): near-failure bins are far more accurate.

subset	true-RUL bin	RMSE	MAE	n
FD001	0-20	3.880	3.130	13
FD001	20-50	4.600	4.020	17
FD001	50-125	14.500	11.320	70
FD002	0-20	3.430	2.830	53
FD002	20-50	7.940	6.290	34
FD002	50-125	15.280	11.840	172
FD003	0-20	3.170	2.530	13
FD003	20-50	4.240	3.670	16
FD003	50-125	15.440	11.180	71
FD004	0-20	4.240	3.350	40
FD004	20-50	10.310	7.580	39
FD004	50-125	15.020	10.640	169

Interpretation. The bin table explains the all-cycles result: far-from-failure rows dominate the average but are operationally irrelevant - maintenance acts below RUL 50, where errors are 3-5x smaller.

6.2 Failure-warning classification

Table 8. Per-class precision/recall/F1 with support (last cycle per engine). Critical/positive recall is the safety metric.

subset	task	approach	model	class	precision	recall	f1	support
FD001	3class	direct	logreg	critical	0.867	1.000	0.929	13
FD001	3class	direct	logreg	macro	0.900	0.935	0.916	100
FD001	binary30	direct	logreg	positive	0.926	1.000	0.962	25
FD001	binary30	direct	logreg	macro	0.963	0.987	0.974	100
FD001	3class	direct	rf	critical	0.846	0.846	0.846	13
FD001	3class	direct	rf	macro	0.841	0.834	0.837	100
FD001	binary30	direct	rf	positive	0.913	0.840	0.875	25
FD001	binary30	direct	rf	macro	0.931	0.907	0.918	100
FD001	3class	direct	hgb	critical	0.833	0.769	0.800	13
FD001	3class	direct	hgb	macro	0.852	0.858	0.853	100

subset	task	approach	model	class	precision	recall	f1	support
FD001	binary30	direct	hgb	positive	0.926	1.000	0.962	25
FD001	binary30	direct	hgb	macro	0.963	0.987	0.974	100
FD001	3class	regress	hgb-via-RUL	critical	0.909	0.769	0.833	13
FD001	3class	regress	hgb-via-RUL	macro	0.896	0.868	0.880	100
FD001	binary30	regress	hgb-via-RUL	positive	0.958	0.920	0.939	25
FD001	binary30	regress	hgb-via-RUL	macro	0.966	0.953	0.959	100
FD002	3class	direct	logreg	critical	0.980	0.925	0.951	53
FD002	3class	direct	logreg	macro	0.882	0.932	0.899	259
FD002	binary30	direct	logreg	positive	0.847	1.000	0.917	61
FD002	binary30	direct	logreg	macro	0.924	0.972	0.944	259
FD002	3class	direct	rf	critical	0.962	0.943	0.952	53
FD002	3class	direct	rf	macro	0.911	0.924	0.917	259
FD002	binary30	direct	rf	positive	0.938	0.984	0.960	61
FD002	binary30	direct	rf	macro	0.966	0.982	0.974	259
FD002	3class	direct	hgb	critical	0.980	0.906	0.941	53
FD002	3class	direct	hgb	macro	0.920	0.942	0.928	259
FD002	binary30	direct	hgb	positive	0.938	1.000	0.968	61
FD002	binary30	direct	hgb	macro	0.969	0.990	0.979	259
FD002	3class	regress	hgb-via-RUL	critical	0.943	0.943	0.943	53
FD002	3class	regress	hgb-via-RUL	macro	0.918	0.918	0.918	259
FD002	binary30	regress	hgb-via-RUL	positive	0.967	0.967	0.967	61
FD002	binary30	regress	hgb-via-RUL	macro	0.979	0.979	0.979	259
FD003	3class	direct	logreg	critical	0.867	1.000	0.929	13
FD003	3class	direct	logreg	macro	0.844	0.925	0.878	100
FD003	binary30	direct	logreg	positive	0.952	1.000	0.976	20
FD003	binary30	direct	logreg	macro	0.976	0.994	0.985	100
FD003	3class	direct	rf	critical	1.000	1.000	1.000	13
FD003	3class	direct	rf	macro	0.932	0.944	0.938	100
FD003	binary30	direct	rf	positive	0.909	1.000	0.952	20
FD003	binary30	direct	rf	macro	0.955	0.988	0.970	100
FD003	3class	direct	hgb	critical	0.867	1.000	0.929	13
FD003	3class	direct	hgb	macro	0.897	0.944	0.918	100
FD003	binary30	direct	hgb	positive	0.952	1.000	0.976	20
FD003	binary30	direct	hgb	macro	0.976	0.994	0.985	100
FD003	3class	regress	hgb-via-RUL	critical	1.000	0.846	0.917	13
FD003	3class	regress	hgb-via-RUL	macro	0.940	0.923	0.928	100
FD003	binary30	regress	hgb-via-RUL	positive	0.947	0.900	0.923	20
FD003	binary30	regress	hgb-via-RUL	macro	0.961	0.944	0.952	100
FD004	3class	direct	logreg	critical	0.973	0.900	0.935	40
FD004	3class	direct	logreg	macro	0.890	0.926	0.901	248
FD004	binary30	direct	logreg	positive	0.823	0.962	0.887	53
FD004	binary30	direct	logreg	macro	0.906	0.953	0.926	248
FD004	3class	direct	rf	critical	1.000	0.925	0.961	40
FD004	3class	direct	rf	macro	0.922	0.921	0.921	248
FD004	binary30	direct	rf	positive	0.923	0.906	0.914	53
FD004	binary30	direct	rf	macro	0.949	0.943	0.946	248
FD004	3class	direct	hgb	critical	1.000	0.900	0.947	40
FD004	3class	direct	hgb	macro	0.925	0.923	0.923	248
FD004	binary30	direct	hgb	positive	0.877	0.943	0.909	53
FD004	binary30	direct	hgb	macro	0.931	0.954	0.942	248
FD004	3class	regress	hgb-via-RUL	critical	1.000	0.925	0.961	40

subset	task	approach	model	class	precision	recall	f1	support
FD004	3class	regress	hgb-via-RUL	macro	0.938	0.919	0.928	248
FD004	binary30	regress	hgb-via-RUL	positive	0.920	0.868	0.893	53
FD004	binary30	regress	hgb-via-RUL	macro	0.942	0.924	0.933	248

Interpretation. Direct classifiers and regress-then-threshold are within noise of each other on macro-F1; the regress-then-threshold path wins 3-class macro-F1 on FD001 (0.880 vs 0.853 for direct HGB) because the RUL ordering regularises boundaries. Small test sets (13 critical engines on FD001/FD003) mean one engine shifts recall by ~8 points - hence the bootstrap CIs below.

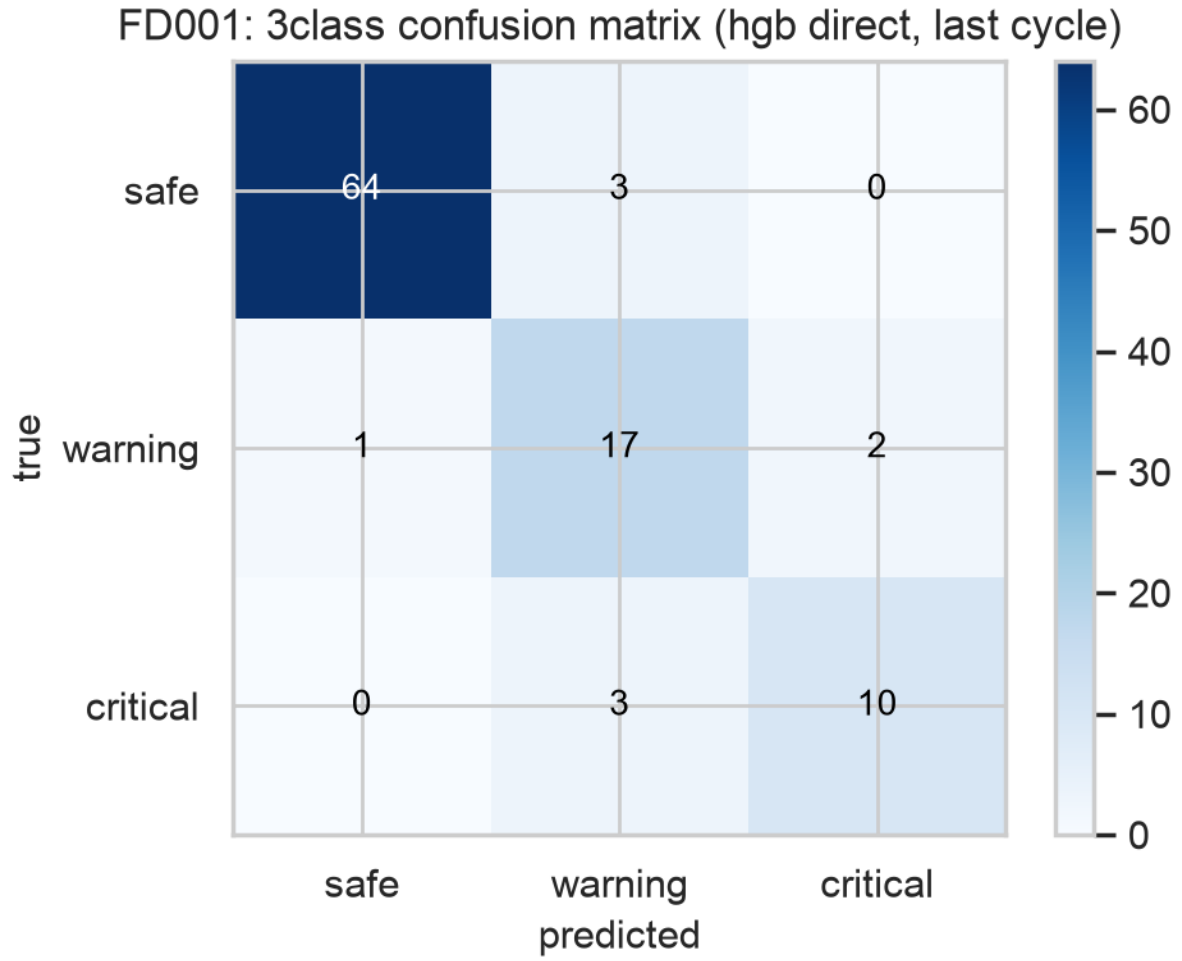


Figure 13. FD001 3-class confusion matrix (hgb direct). Off-diagonal mass sits in adjacent classes; dangerous safe?critical confusions are counted in the text.

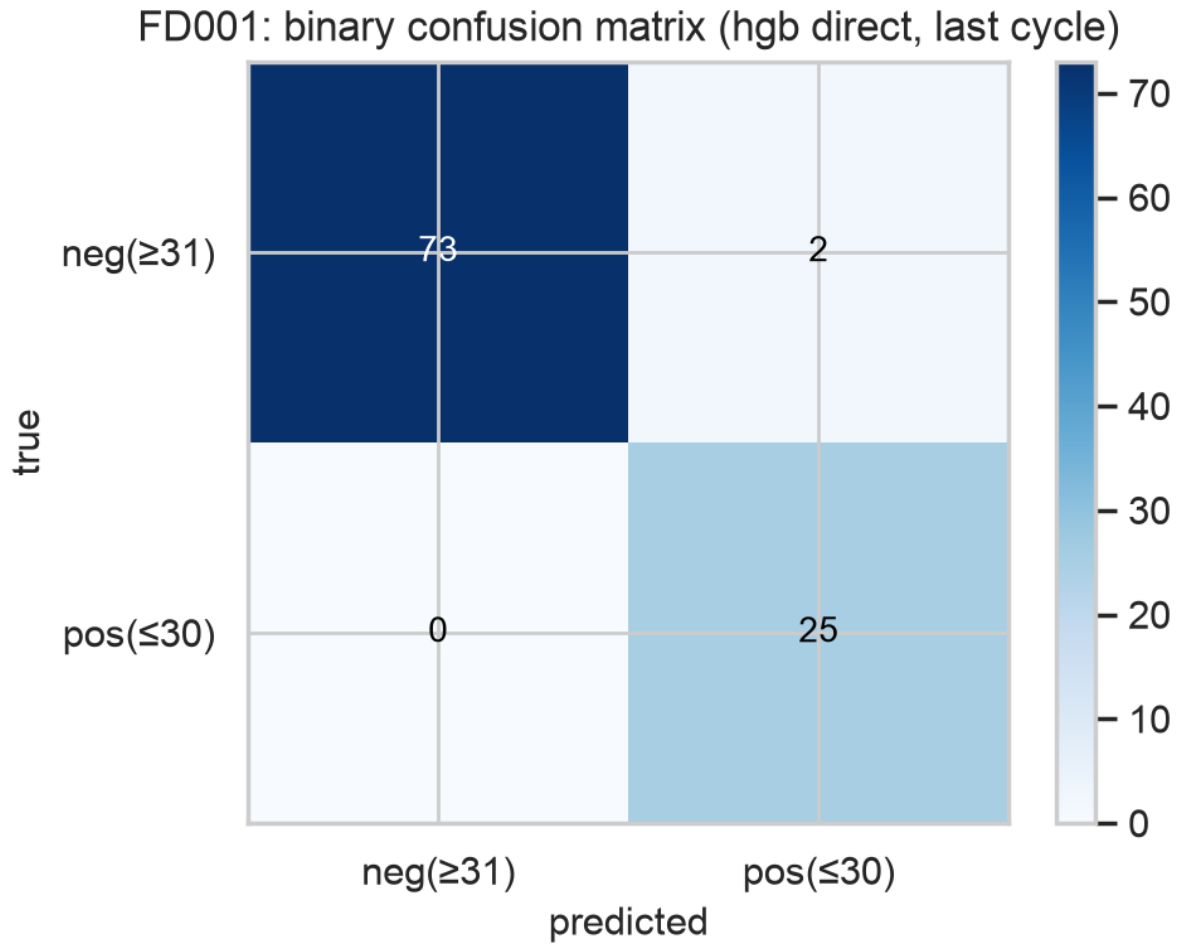


Figure 14. FD001 binary <=30 confusion matrix (hgb direct).

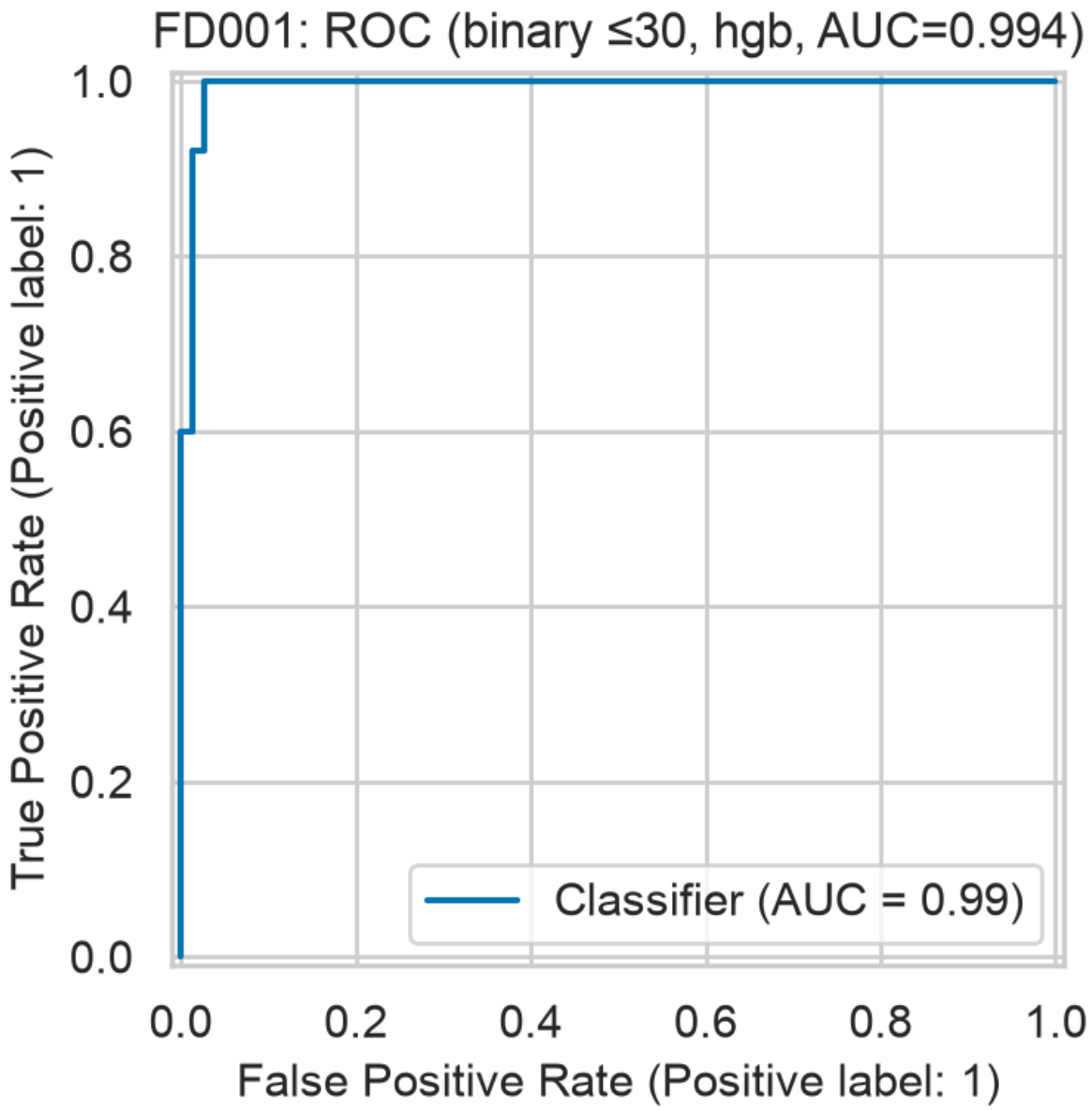


Figure 15. FD001 ROC for fails-within-30 (hgb).

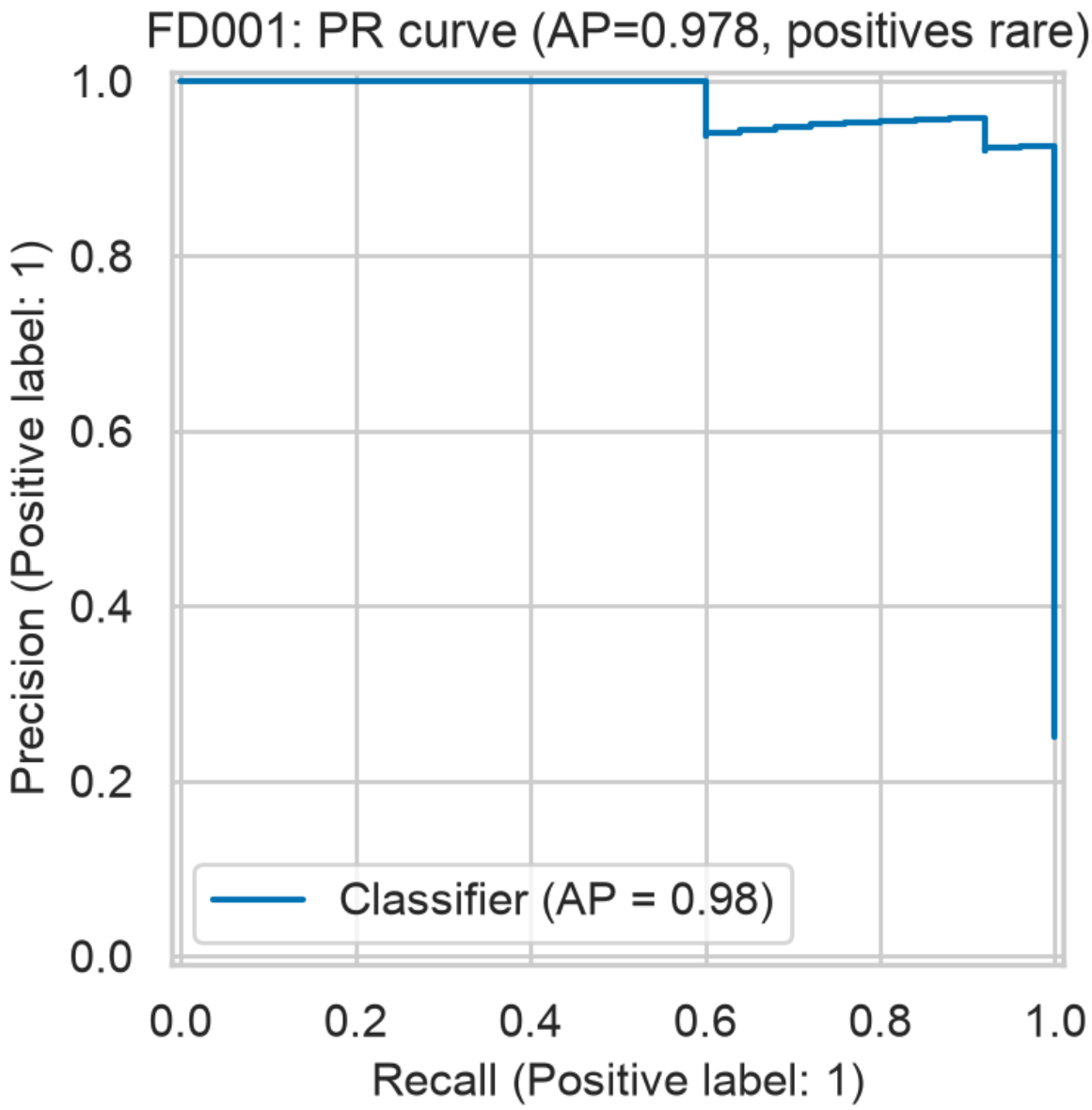


Figure 16. FD001 precision-recall curve (hgb): positives are rare, so PR matters more than ROC.

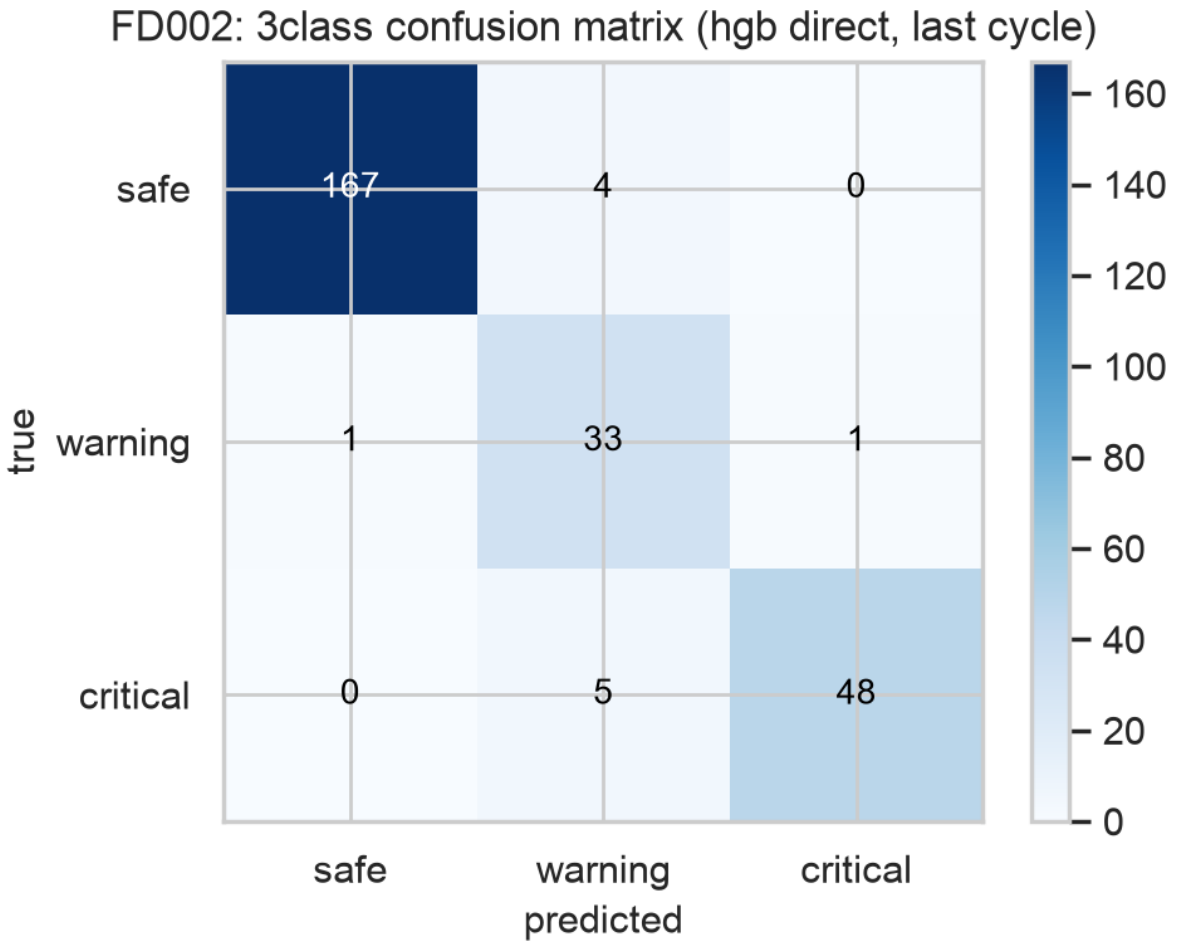


Figure 17. FD002 3-class confusion matrix (hgb direct). Off-diagonal mass sits in adjacent classes; dangerous safe?critical confusions are counted in the text.

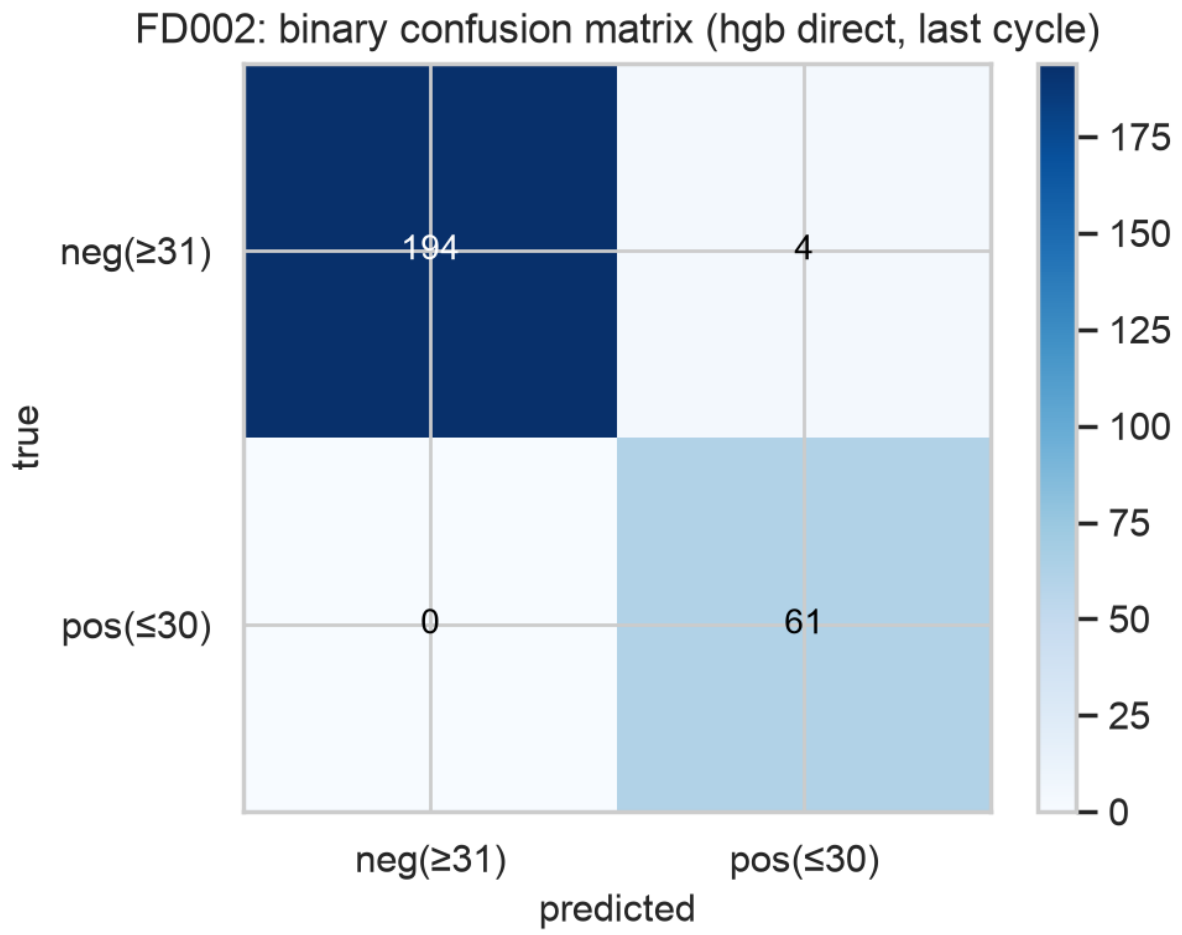


Figure 18. FD002 binary <=30 confusion matrix (hgb direct).

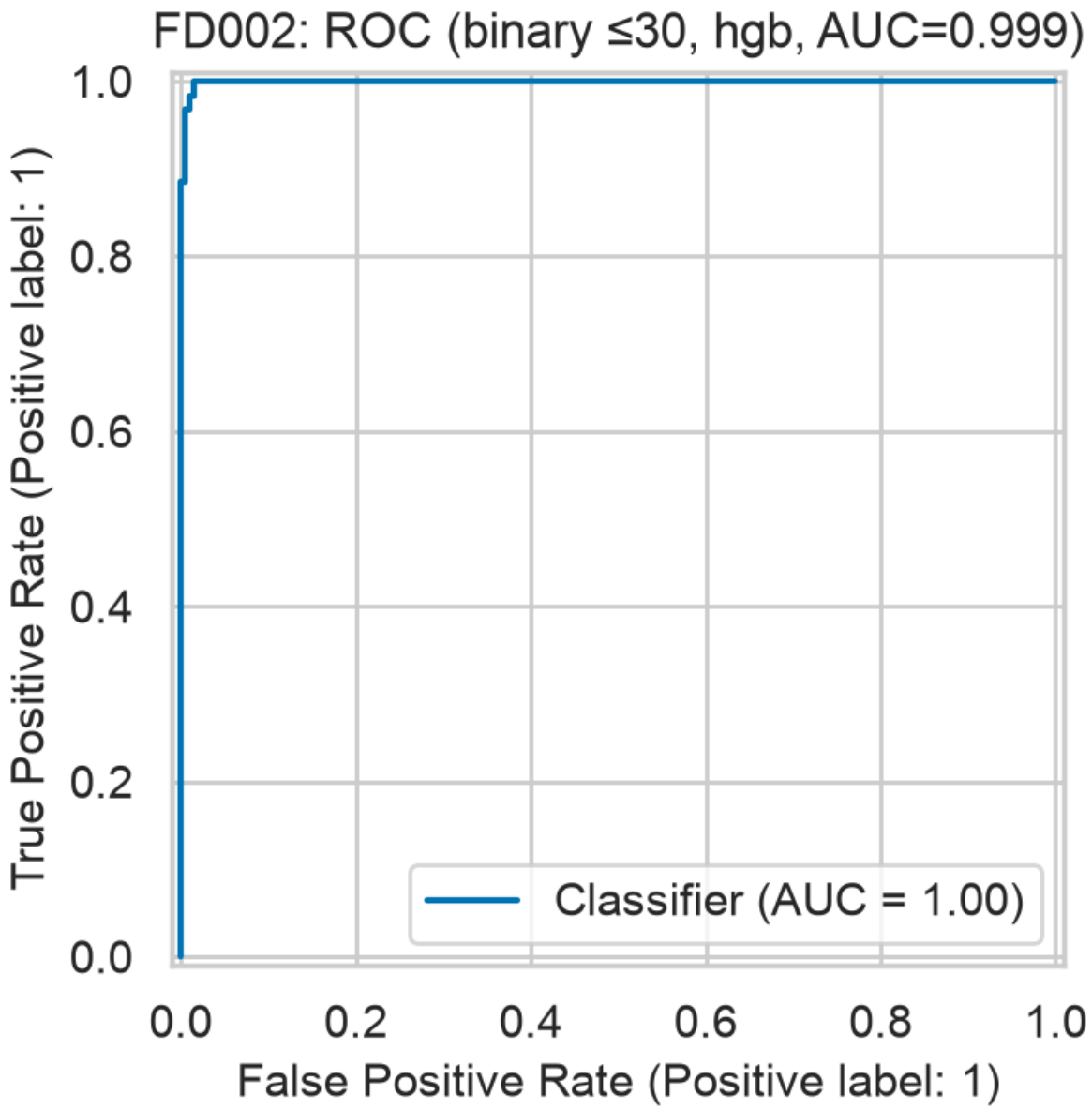


Figure 19. FD002 ROC for fails-within-30 (hgb).

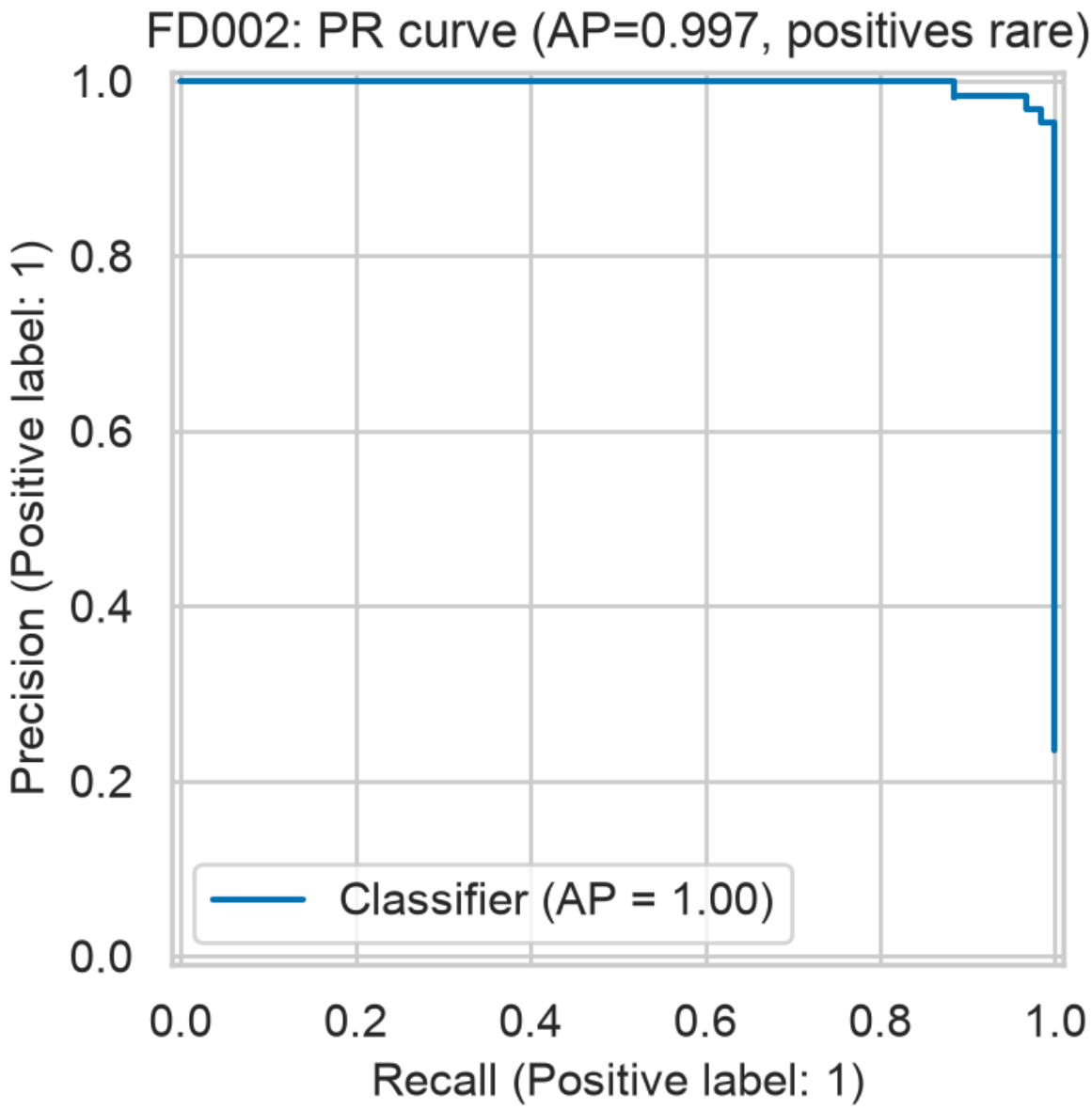


Figure 20. FD002 precision-recall curve (hgb): positives are rare, so PR matters more than ROC.

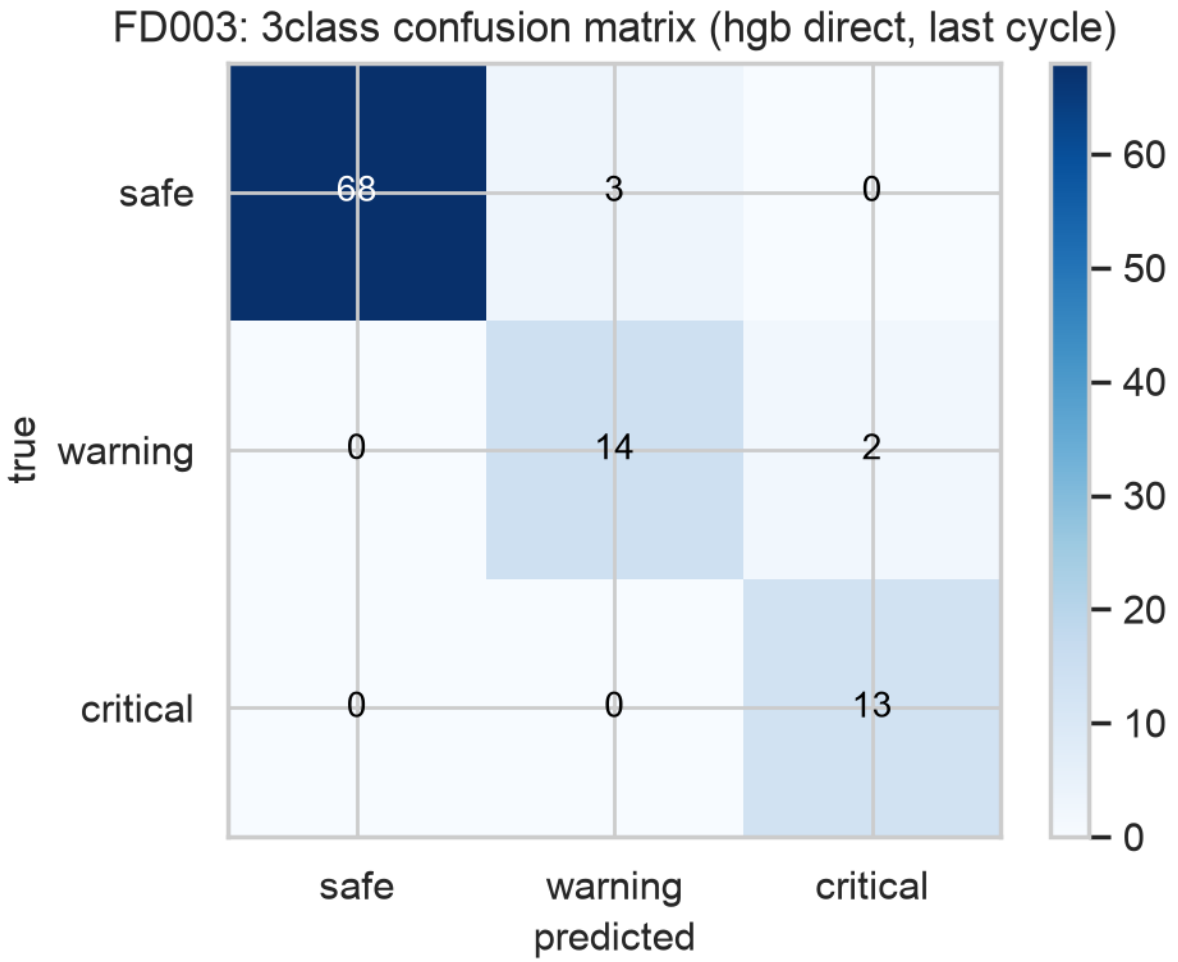


Figure 21. FD003 3-class confusion matrix (hgb direct). Off-diagonal mass sits in adjacent classes; dangerous safe?critical confusions are counted in the text.

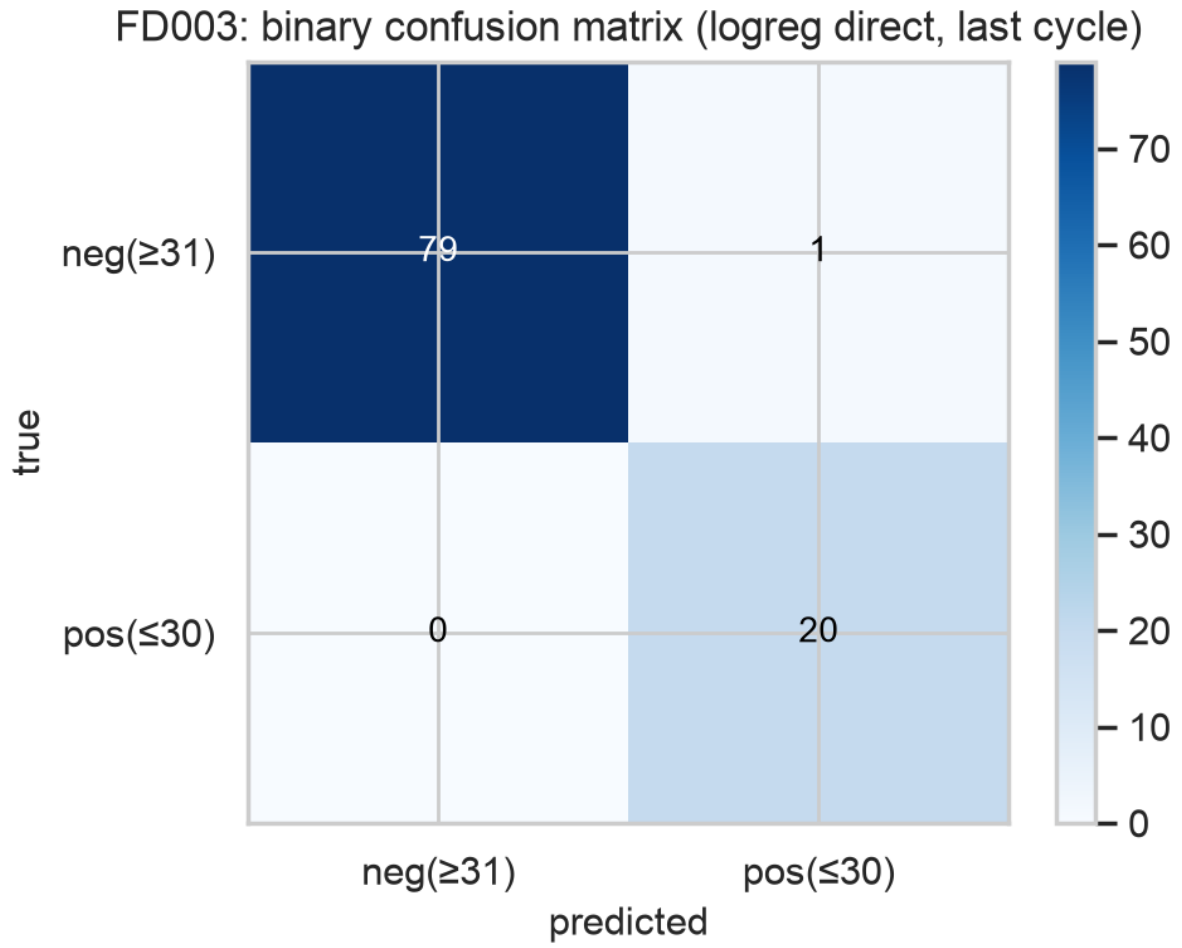


Figure 22. FD003 binary <=30 confusion matrix (logreg direct).

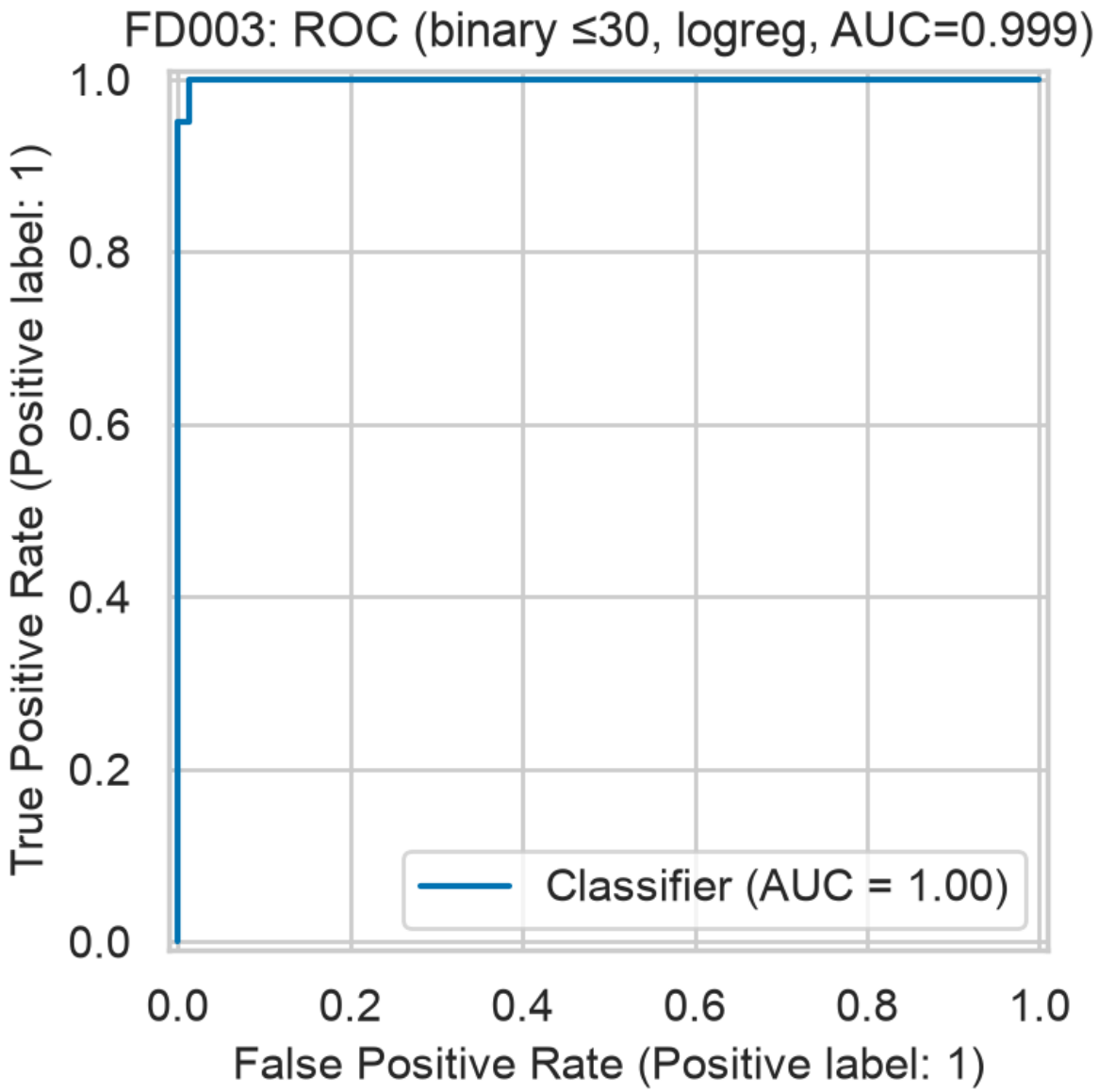


Figure 23. FD003 ROC for fails-within-30 (logreg).

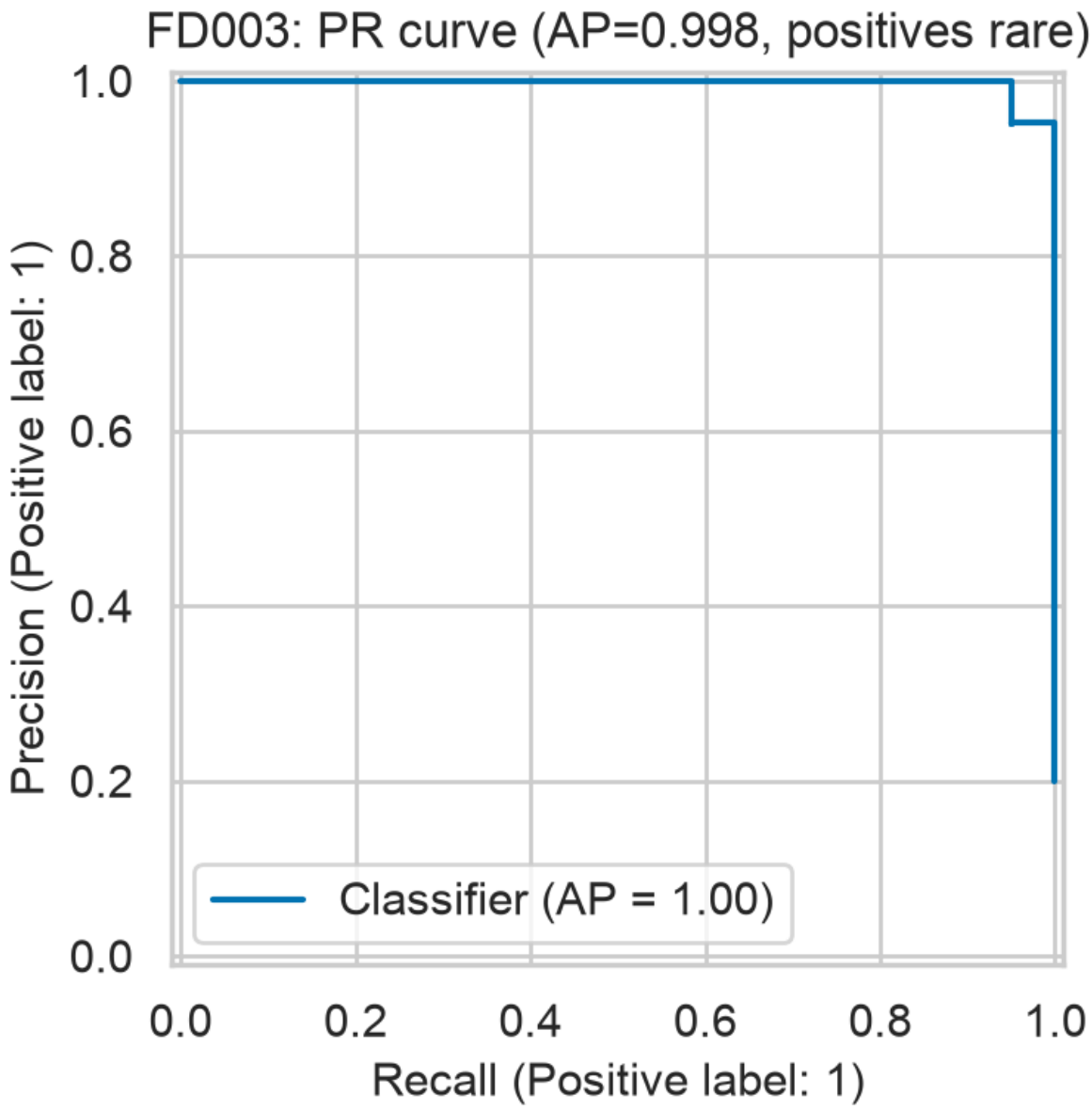


Figure 24. FD003 precision-recall curve (logreg): positives are rare, so PR matters more than ROC.

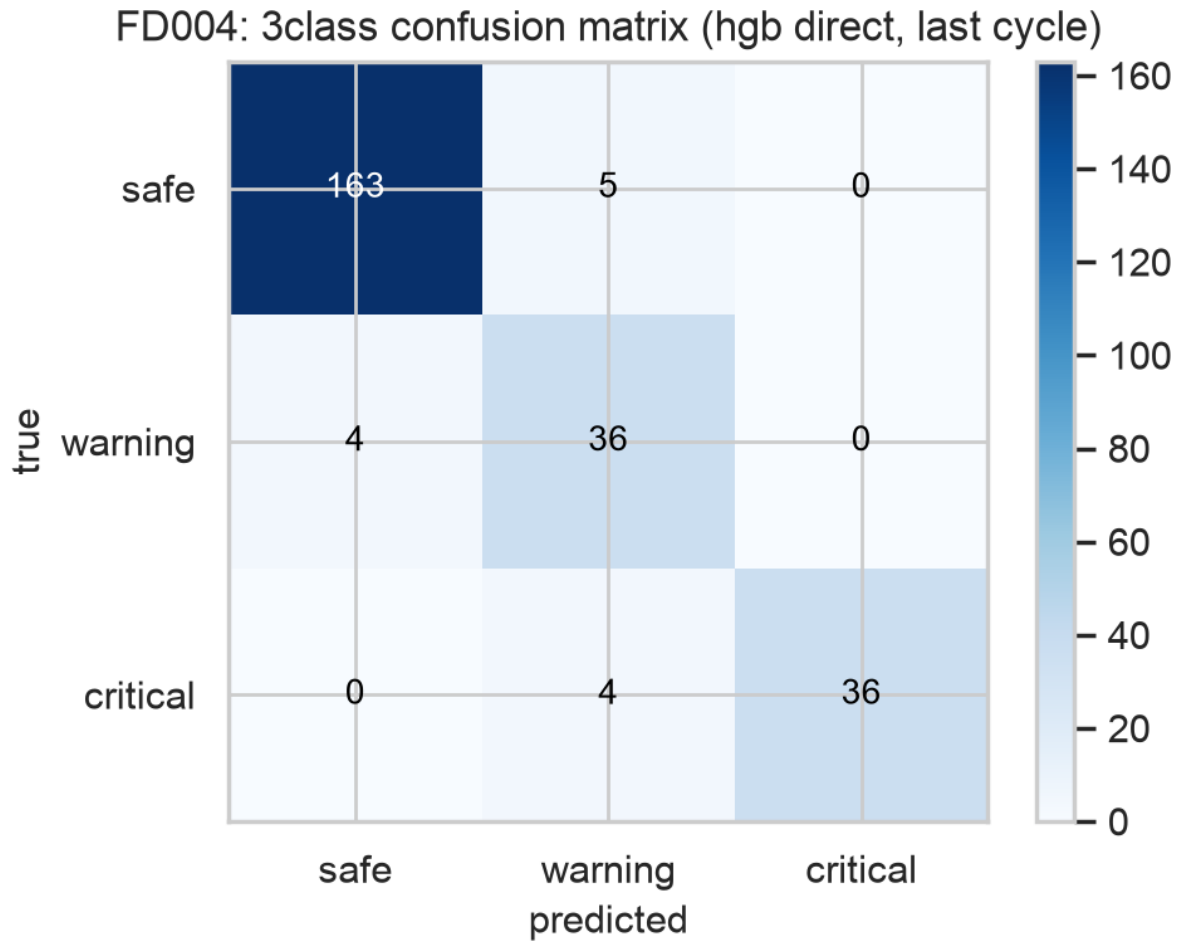


Figure 25. FD004 3-class confusion matrix (hgb direct). Off-diagonal mass sits in adjacent classes; dangerous safe?critical confusions are counted in the text.

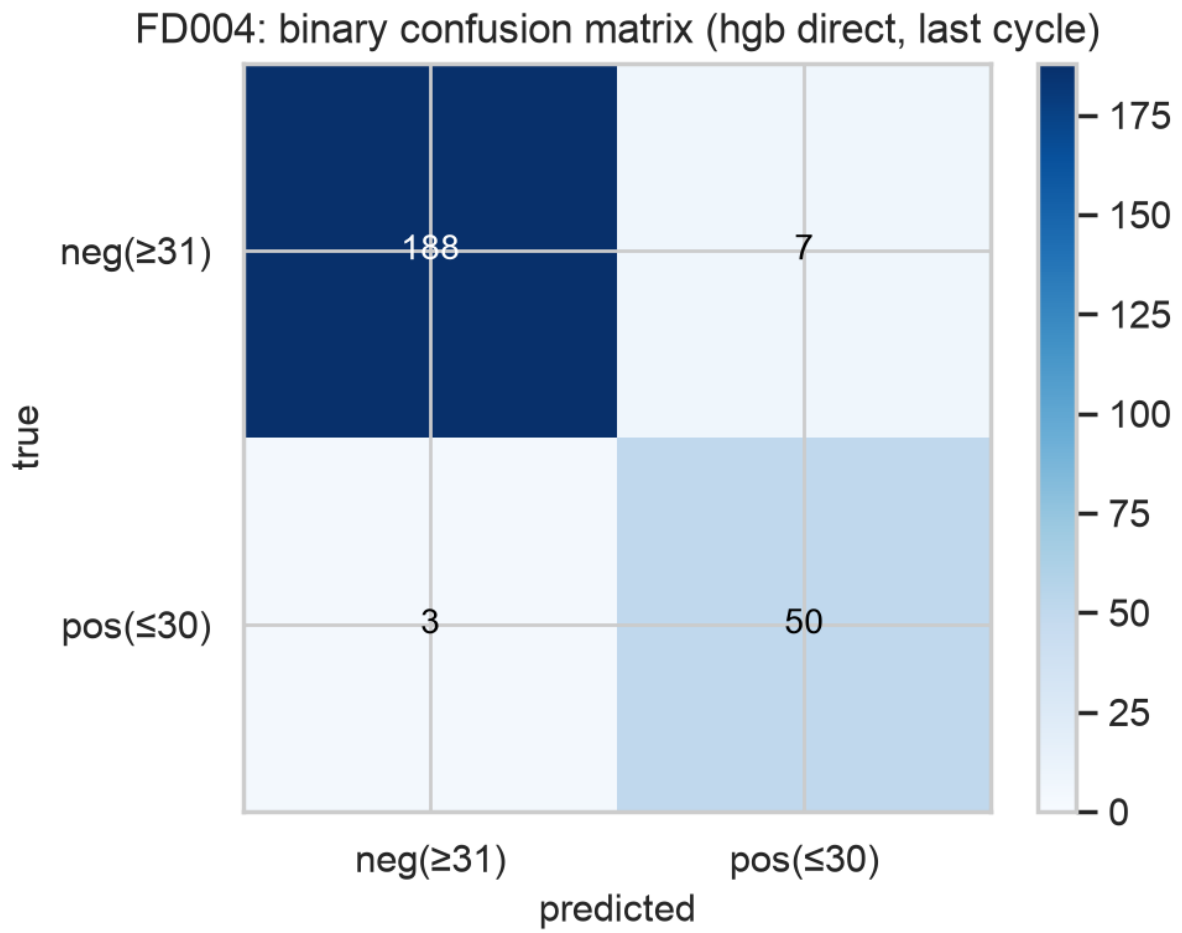


Figure 26. FD004 binary <=30 confusion matrix (hgb direct).

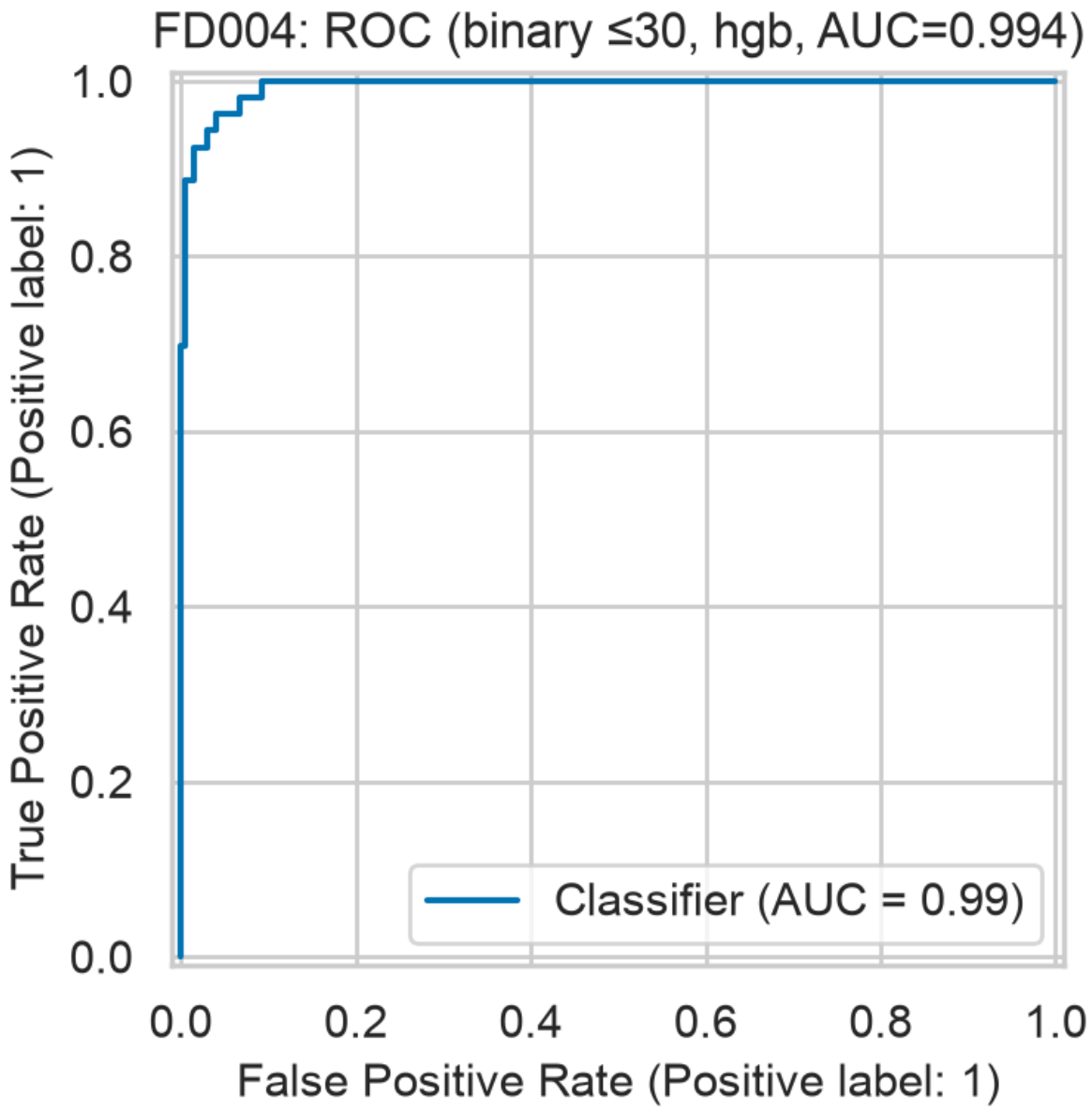


Figure 27. FD004 ROC for fails-within-30 (hgb).

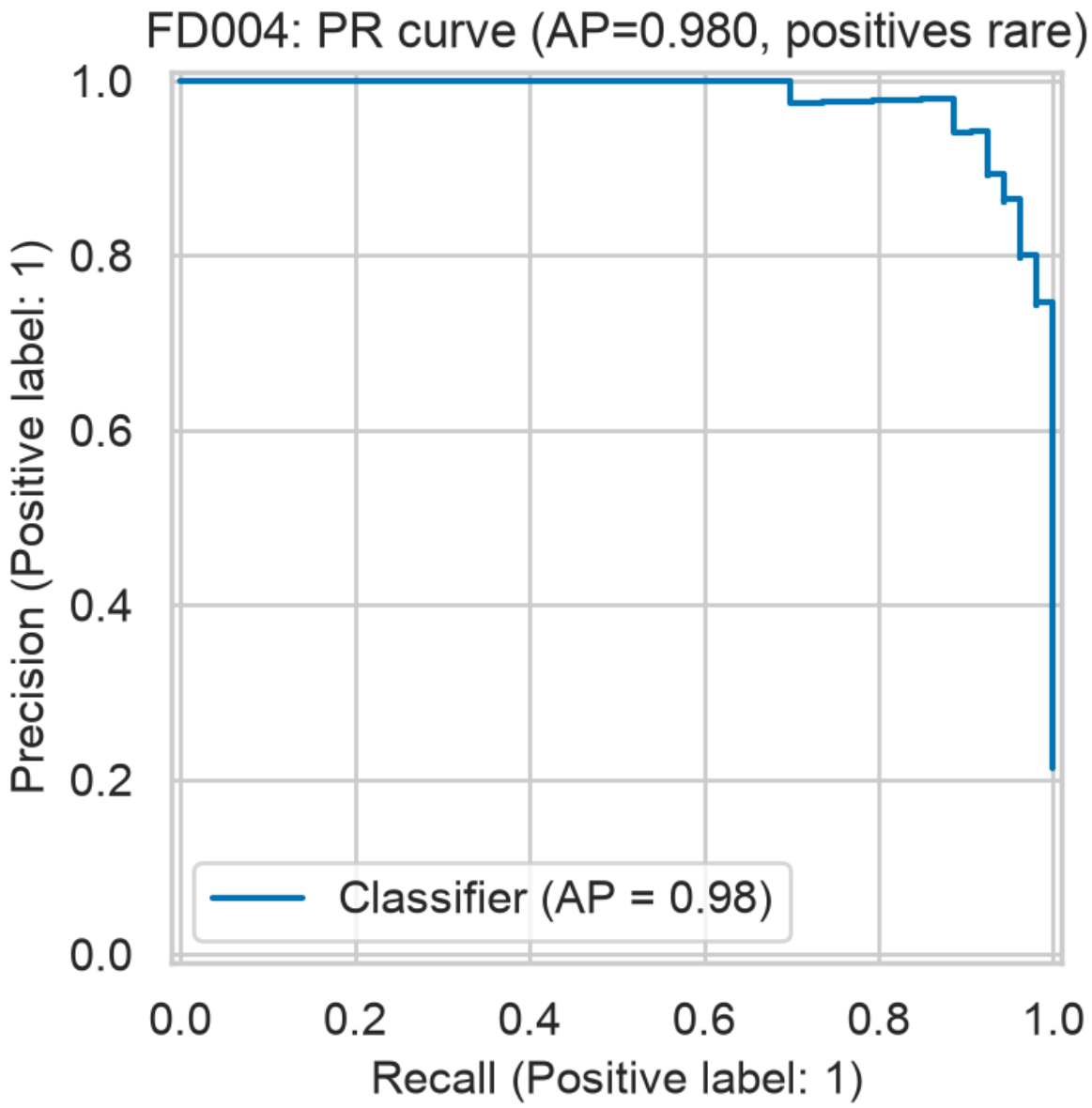


Figure 28. FD004 precision-recall curve (hgb): positives are rare, so PR matters more than ROC.

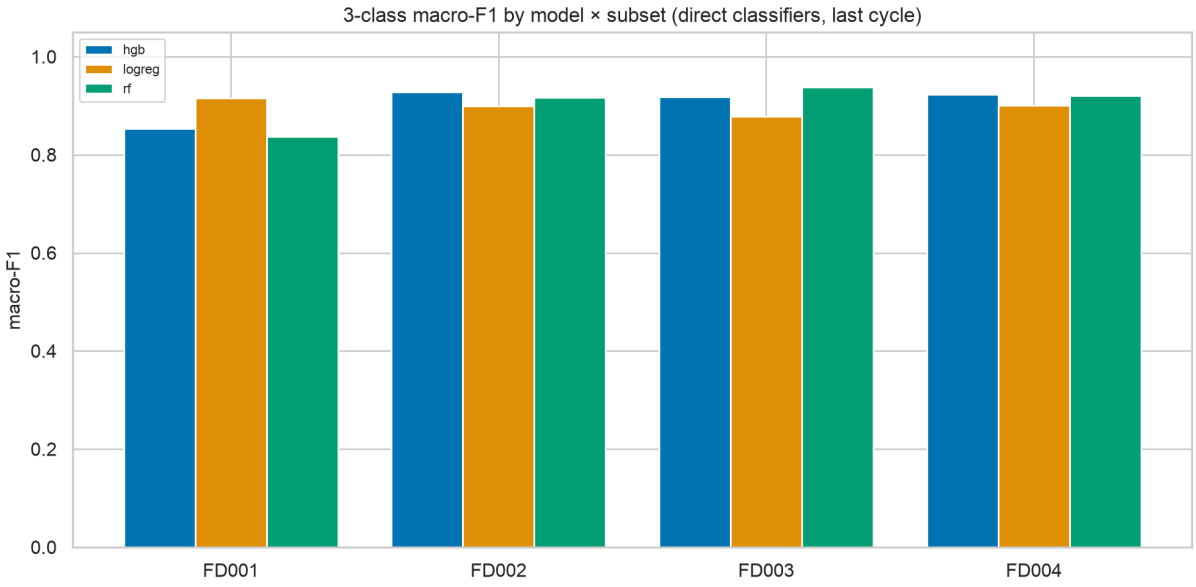


Figure 29. 3-class macro-F1 across models and subsets (direct).

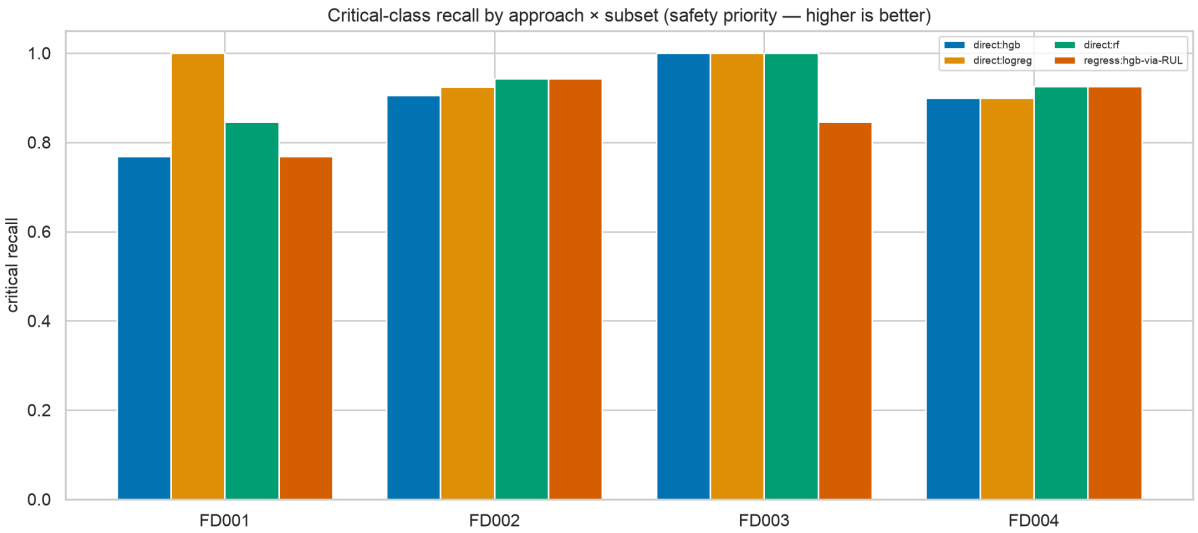


Figure 30. Critical recall across approaches and subsets.

6.3 Threshold policy, lead time and cost

Table 9. Frozen thresholds from OOF predictions at truncation points; test recall/precision with bootstrap CIs; expected cost (assumed 10:1) at t=30 vs tuned t.

subset	frozen t	rule	target met	test recall	test precision	cost 30->tuned
FD001	40.000	min cost subject to OOF recall>=0.95	True	1.000 [1.00,1.00]	0.926	21->2
FD002	40.000	min cost subject to OOF recall>=0.95	True	1.000 [1.00,1.00]	0.803	22->15
FD003	34.000	min cost subject to OOF recall>=0.95	True	1.000 [1.00,1.00]	0.909	21->2
FD004	48.000	min cost subject to OOF recall>=0.95	True	1.000 [1.00,1.00]	0.716	74->21

Interpretation. The 0.95 OOF recall target was feasible on all four subsets, so every frozen threshold minimises cost subject to high recall; test recall is 1.00 everywhere at precisions 0.72-0.93. FD004 pays the most false alarms (lowest precision) - the price of full recall under two fault modes.

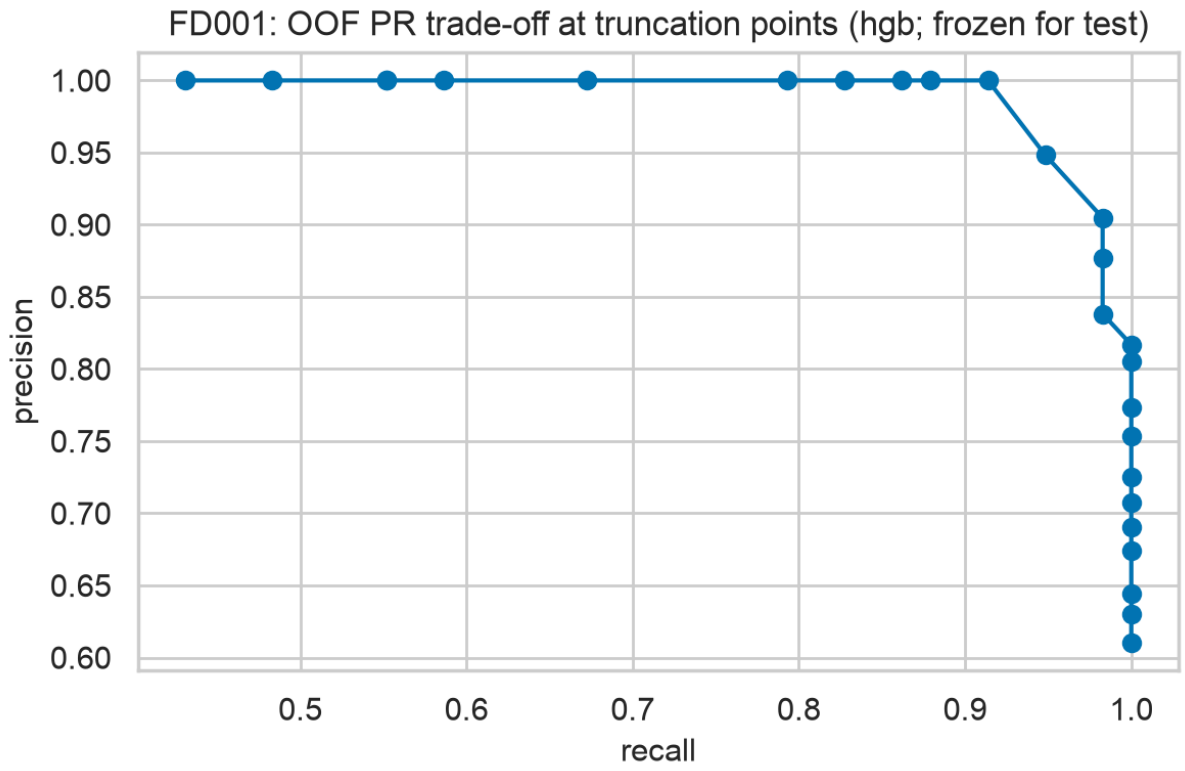


Figure 31. FD001 OOF precision-recall trade-off over candidate thresholds (validation only).

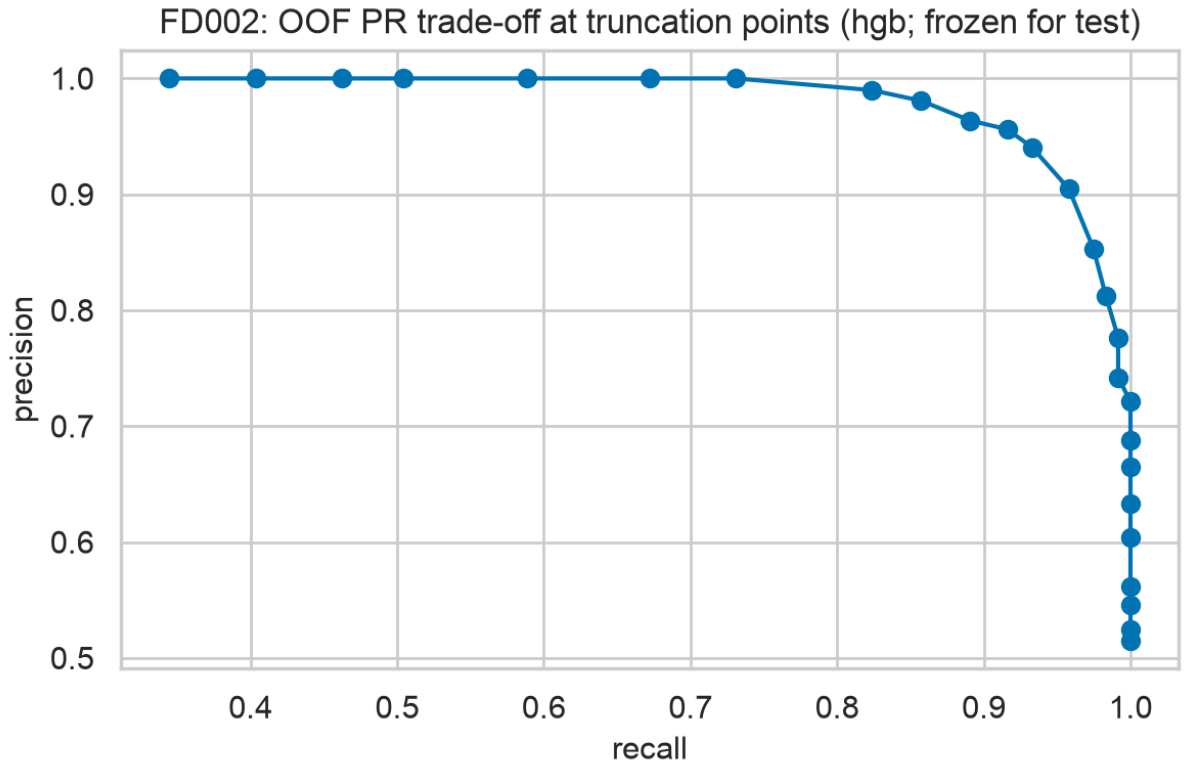


Figure 32. FD002 OOF precision-recall trade-off over candidate thresholds (validation only).

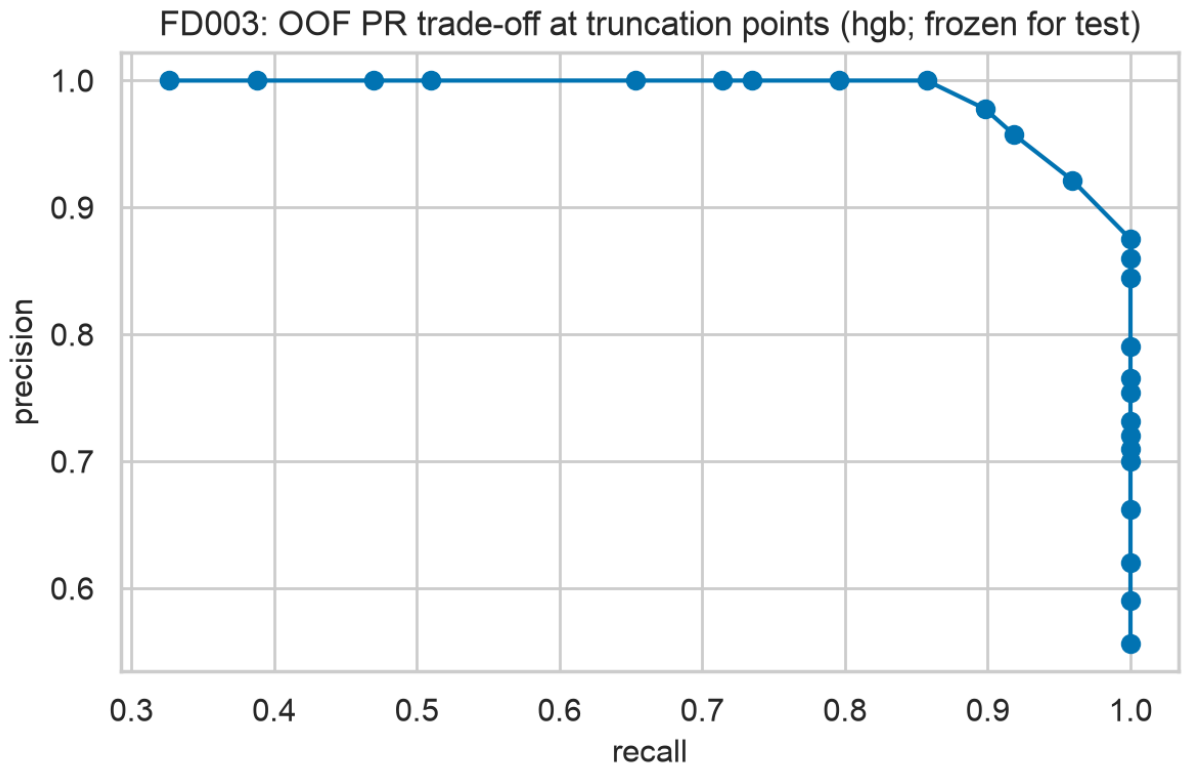


Figure 33. FD003 OOF precision-recall trade-off over candidate thresholds (validation only).

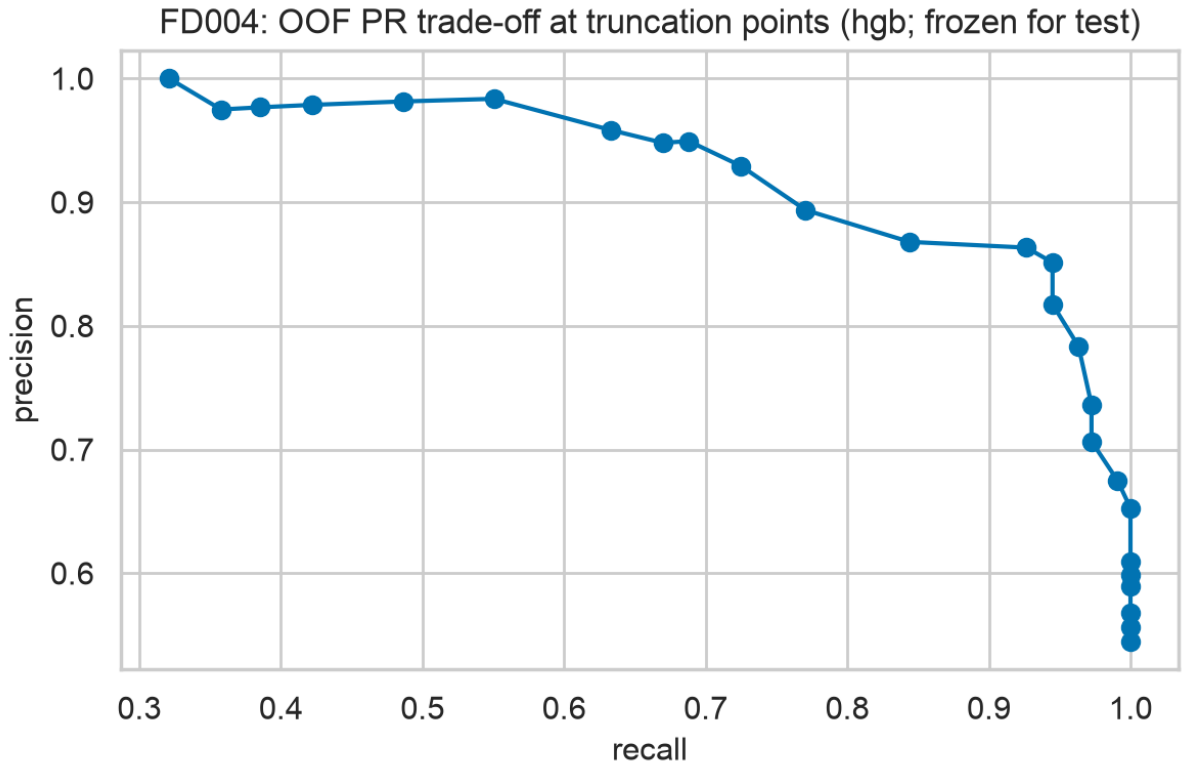


Figure 34. FD004 OOF precision-recall trade-off over candidate thresholds (validation only).

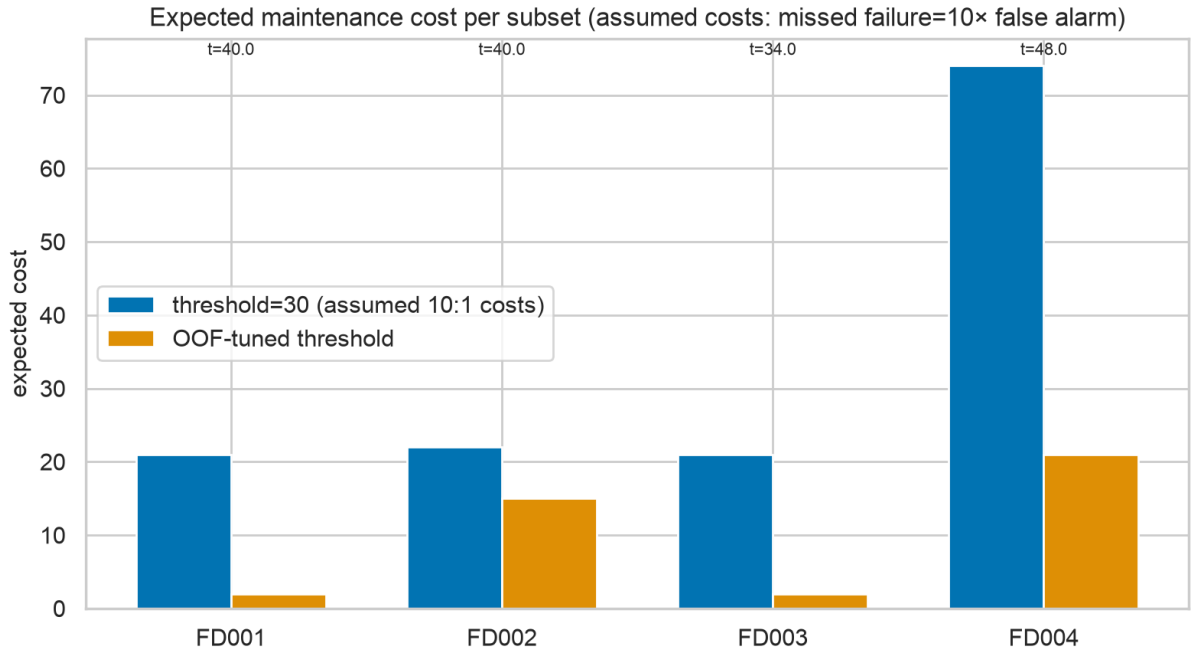


Figure 35. Expected cost at default t=30 vs OOF-tuned t (assumed 10:1 costs).

Table 10. Lead times (cycles of advance warning), premature-alert fractions, false alarms per 100 cycles, any-time critical detection.

subset	warn lead med[IQR]	crit lead med[IQR]	premature warn/crit	FAR/100	any-time crit
FD001	51[48,56]	21[20,24]	0.17/0.00	0.670	10/13
FD002	53[45,62]	19[17,22]	0.30/0.04	0.960	51/53
FD003	50[48,54]	19[16,21]	0.10/0.00	0.280	11/13
FD004	50[44,55]	19[16,22]	0.16/0.05	0.580	38/40

Interpretation. Median warning lead 51-53 cycles on every subset - comfortably plannable. Any-time detection trails last-cycle recall in general because an engine can be caught mid-history yet misclassified at its final point (both rates are reported side by side above).

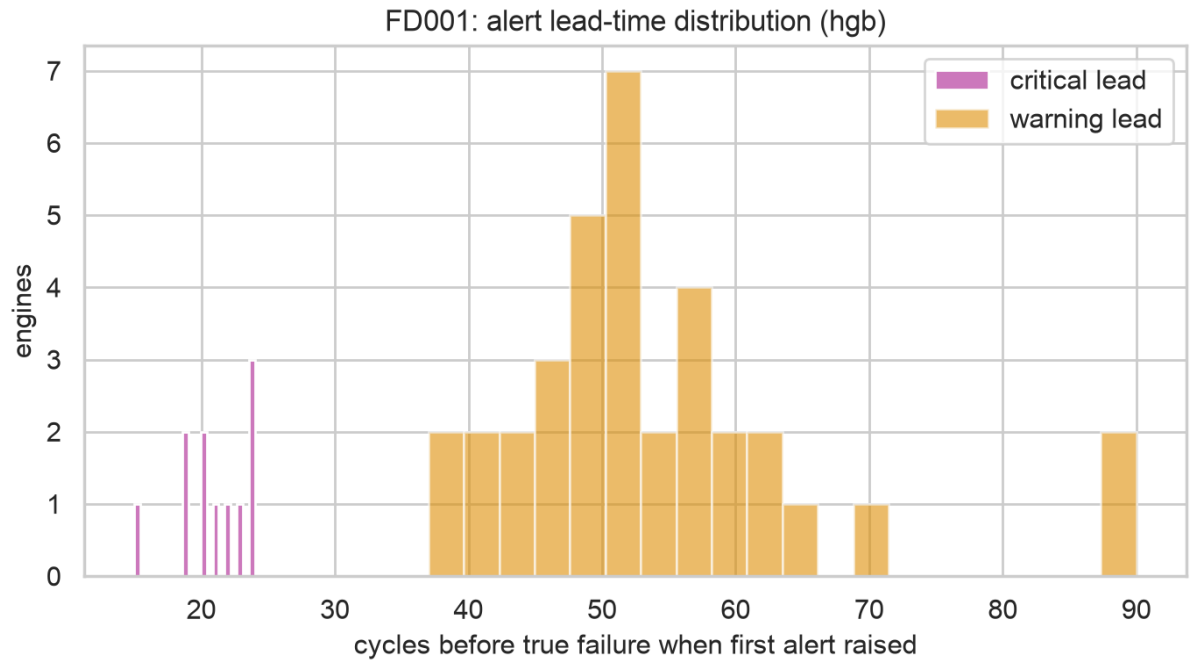


Figure 36. FD001 lead-time distribution (hgb).

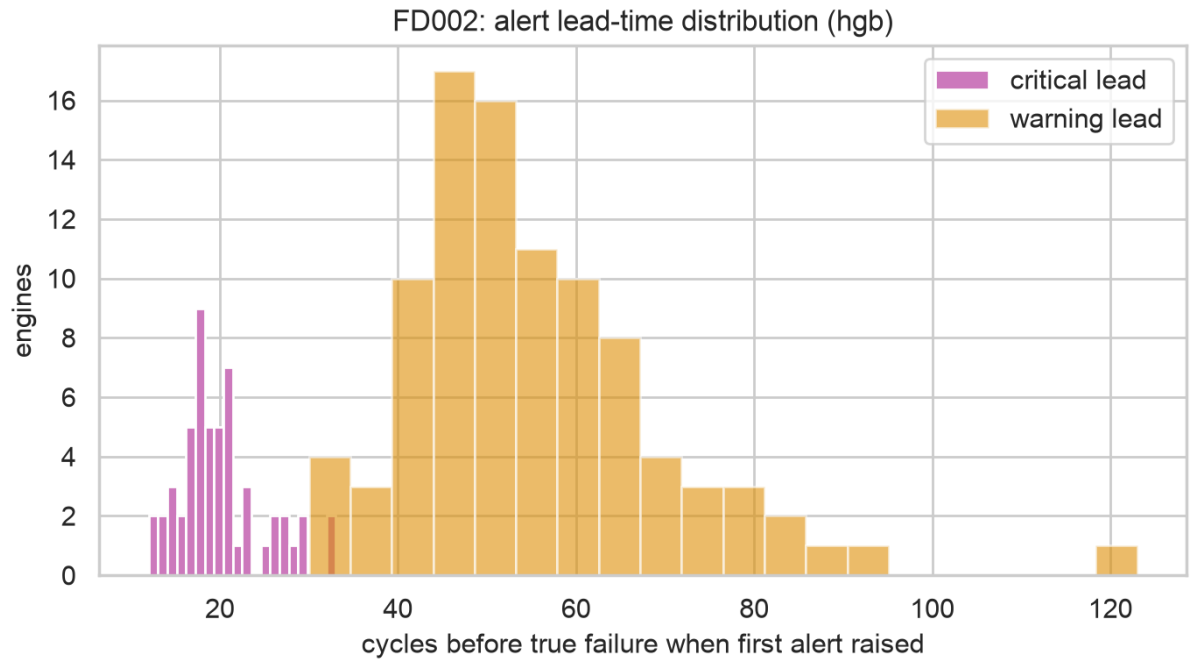


Figure 37. FD002 lead-time distribution (hgb).

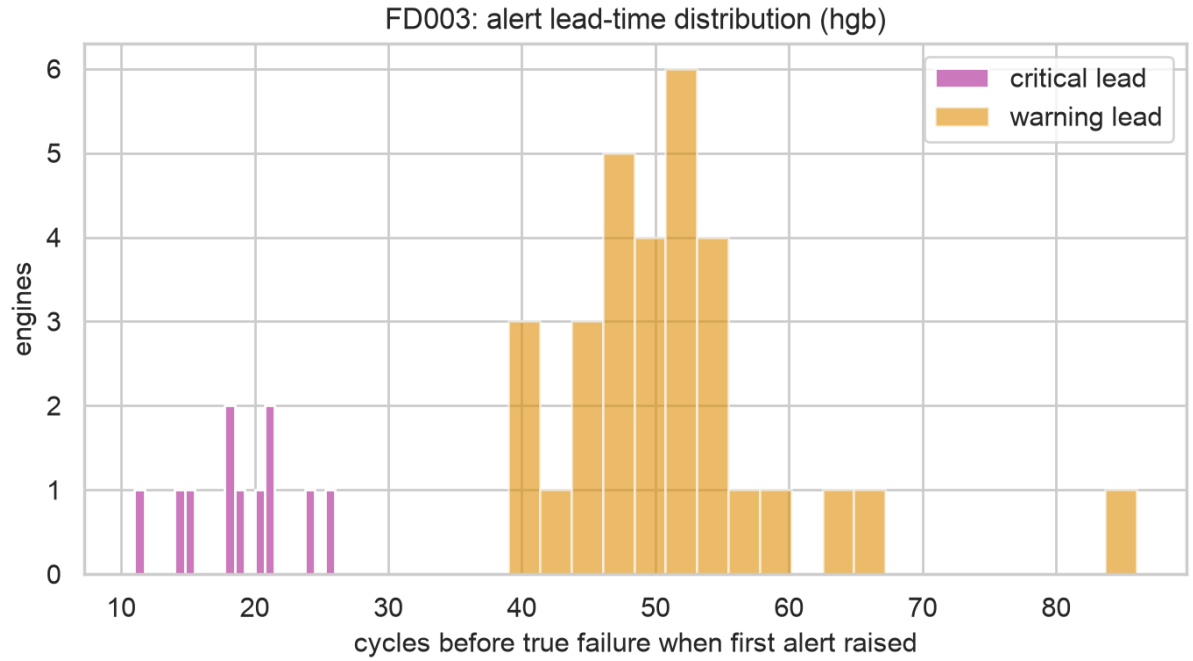


Figure 38. FD003 lead-time distribution (hgb).

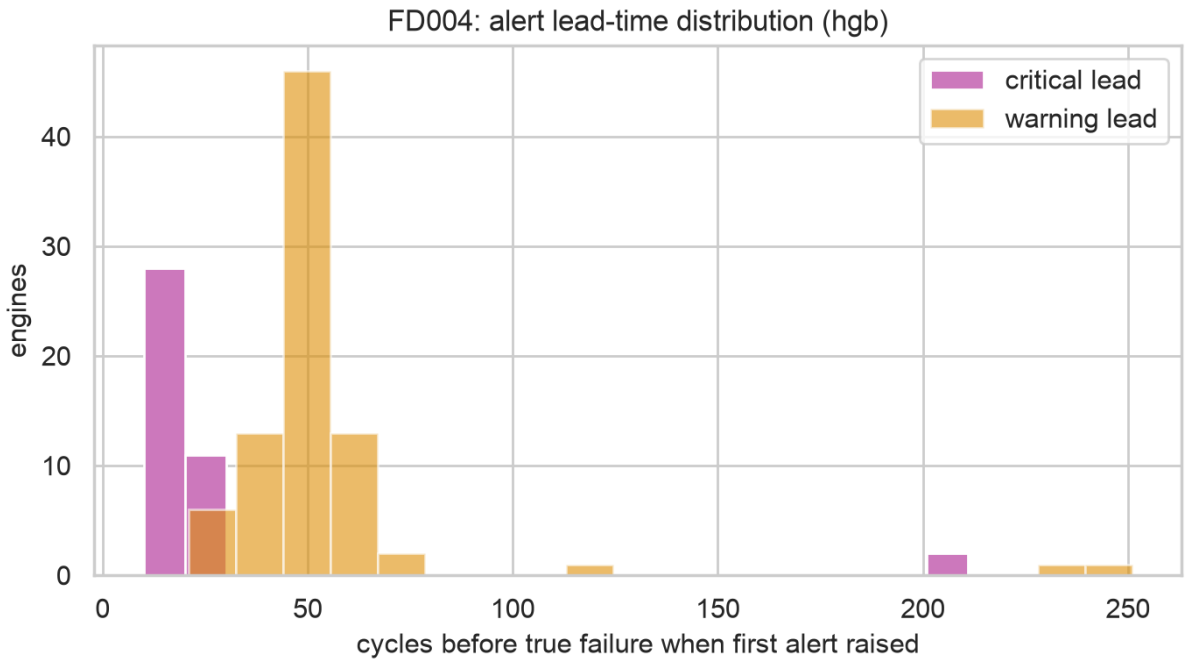


Figure 39. FD004 lead-time distribution (hgb).

6.4 Ablations

Table 11. Ablation study (default HGB, grouped 5-fold CV + frozen test value; cap ablation on a common capped-125 target).

subset	ablation	setting	metric_name	cv_mean	test_value
FD001	baseline	default	rmse_cap	11.720	12.038
FD001	smooth_order	legacy_smooth_first	rmse_cap	11.650	12.369
FD001	smooth_order	fixed_norm_first	rmse_cap	11.720	12.038
FD001	rolling	[10]	rmse_cap	15.230	15.713
FD001	rolling	[10, 30]	rmse_cap	11.720	12.038
FD001	rolling	[5, 10, 30]	rmse_cap	11.640	12.530
FD001	rul_cap	100	rmse_common125	7.590	14.882
FD001	rul_cap	125	rmse_common125	11.710	11.963
FD001	rul_cap	130	rmse_common125	11.640	12.305
FD001	no_cycle	dropped	rmse_cap	12.530	13.248
FD001	no_ops	dropped	rmse_cap	11.720	12.038
FD001	class_weight	balanced	macro_f1	0.920	0.841
FD001	class_weight	nan	macro_f1	0.920	0.914
FD001	r_vs_c	regress_then_threshold	macro_f1	0.910	0.903
FD001	r_vs_c	direct_classifier	macro_f1	0.920	0.841
FD002	baseline	default	rmse_cap	13.110	12.746
FD002	smooth_order	legacy_smooth_first	rmse_cap	18.970	18.111
FD002	smooth_order	fixed_norm_first	rmse_cap	13.110	12.746
FD002	condnorm	off_global	rmse_cap	19.630	18.902
FD002	condnorm	on_cluster	rmse_cap	13.110	12.746
FD002	rolling	[10]	rmse_cap	16.030	14.908
FD002	rolling	[10, 30]	rmse_cap	13.110	12.746
FD002	rolling	[5, 10, 30]	rmse_cap	13.170	12.968
FD002	rul_cap	100	rmse_common125	8.800	16.644
FD002	rul_cap	125	rmse_common125	13.100	12.739
FD002	rul_cap	130	rmse_common125	13.230	13.056
FD002	no_cycle	dropped	rmse_cap	14.440	13.738

subset	ablation	setting	metric_name	cv_mean	test_value
FD002	no_ops	dropped	rmse_cap	13.110	12.746
FD002	class_weight	balanced	macro_f1	0.900	0.934
FD002	class_weight	nan	macro_f1	0.890	0.928
FD002	r_vs_c	regress_then_threshold	macro_f1	0.880	0.907
FD002	r_vs_c	direct_classifier	macro_f1	0.900	0.934

Interpretation. Three findings matter. (1) The v1 order bug is quantified: smoothing before normalisation costs +5.4 RMSE on FD002 (18.11 vs 12.75); condition normalisation on/off costs +6.2 (18.90 vs 12.75). (2) Rolling [10,30] beats [10] alone; adding window 5 adds nothing. (3) Honest negatives: class_weight=balanced hurts test macro-F1 vs None (FD001 0.841 vs 0.914); op-settings features add nothing once normalised (FD001 12.04 both ways); cycle matters (+1.2 without it) and is legitimate observable age, not leakage. Cap 100 is too aggressive; 125 ? 130 on the common target.

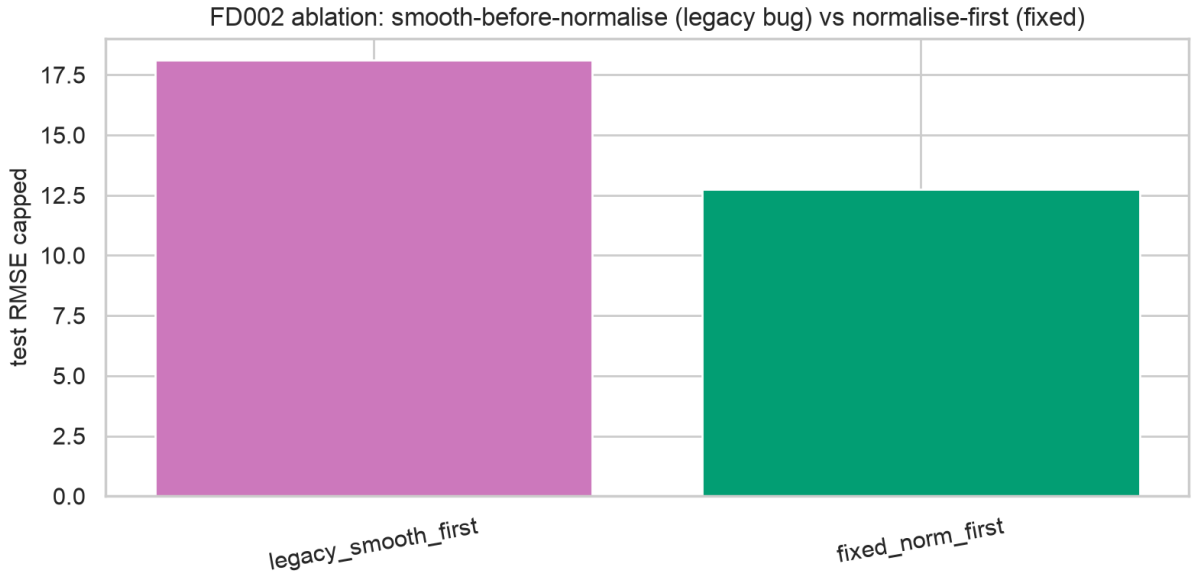


Figure 40. FD002 smooth-order ablation: the v1 bug as a number.

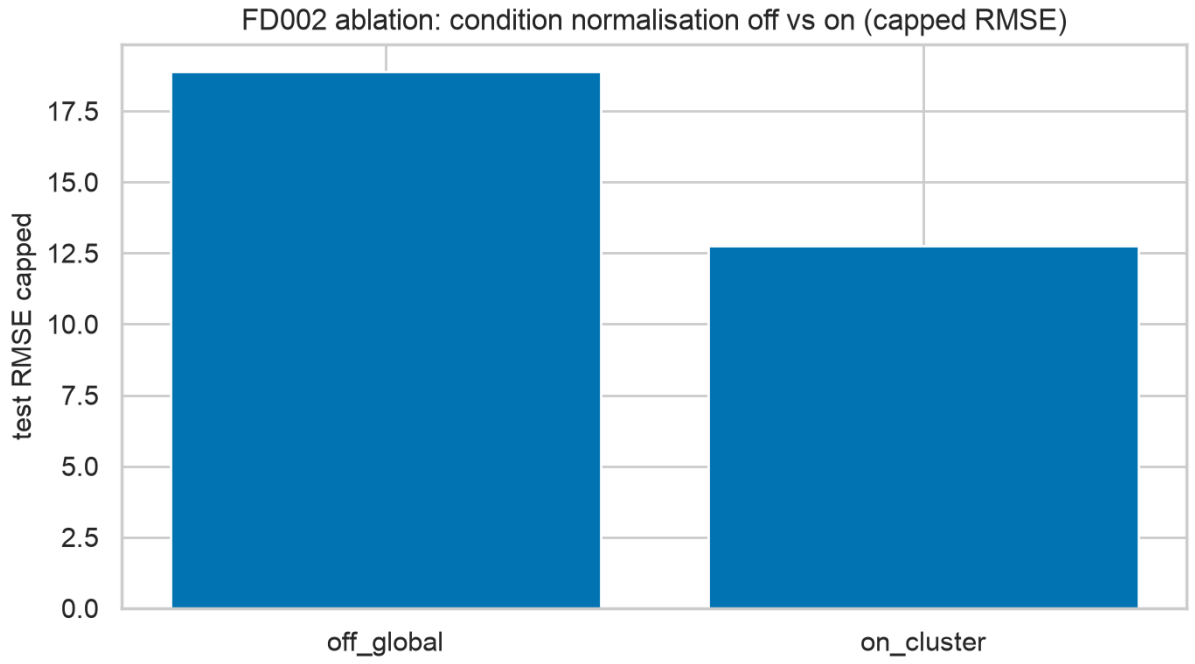


Figure 41. FD002 condition-normalisation on/off.

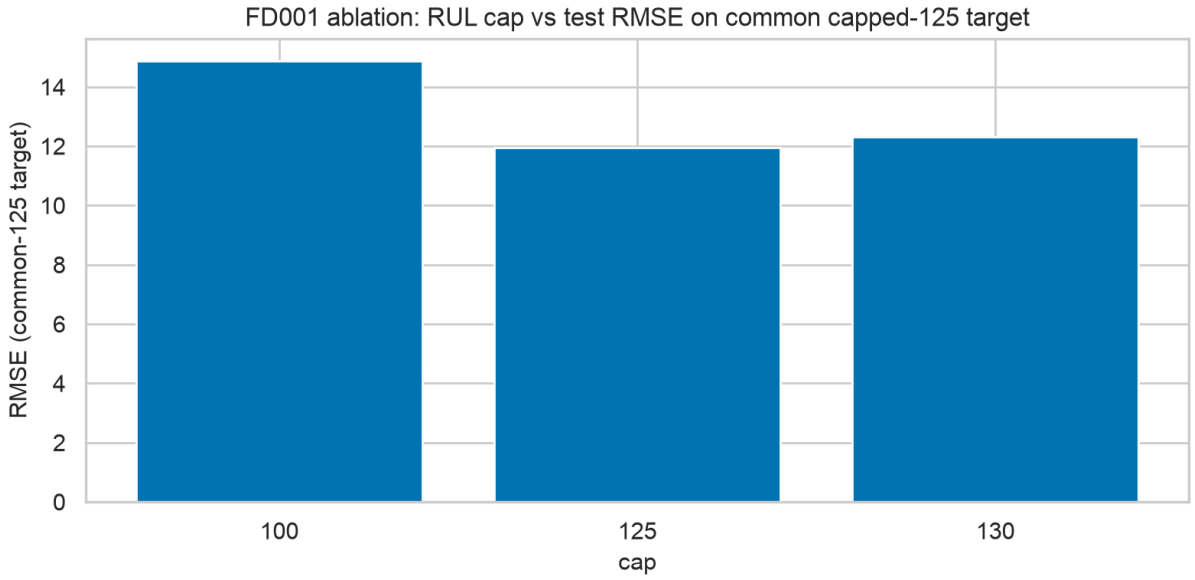


Figure 42. FD001 RUL-cap ablation on the common capped-125 target.

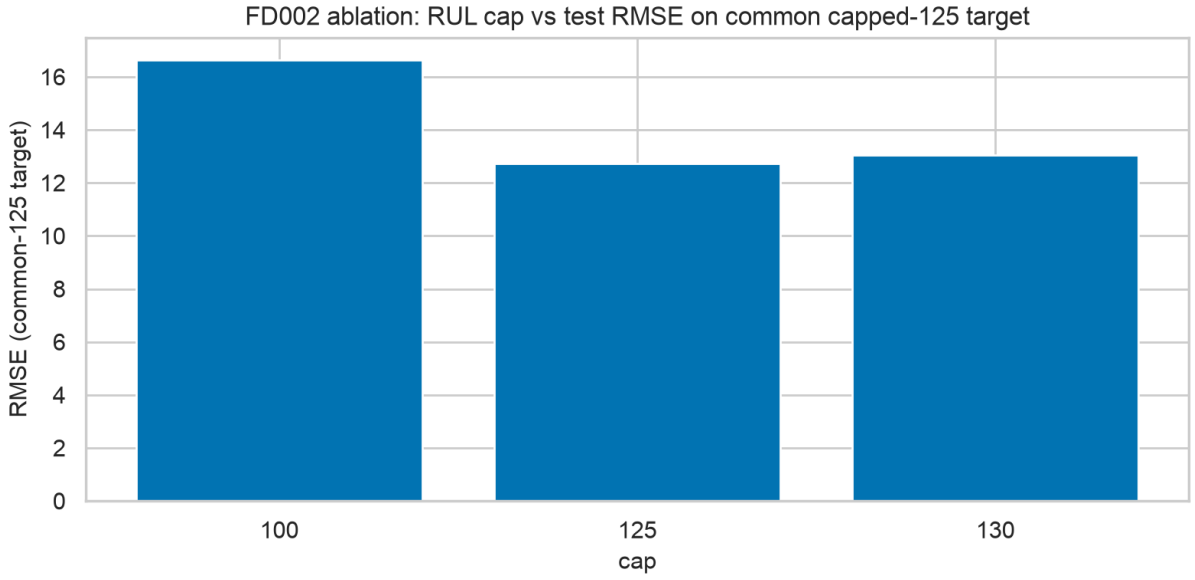


Figure 43. FD002 RUL-cap ablation on the common capped-125 target.

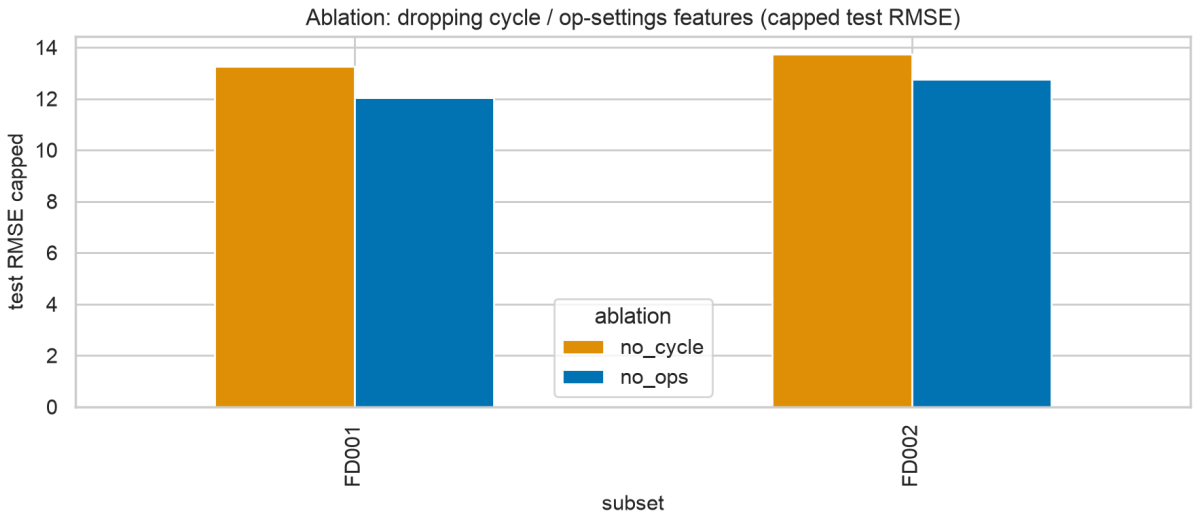


Figure 44. Dropping cycle vs op-settings features.

Table 12. Deep window-length ablation (single seed): 30 is the sweet spot; 50 overfits (FD001 LSTM holdout

10.05 -> test 15.72).

subset	model	window	holdout_rmse	test_rmse_cap
FD001	lstm	20	13.760	15.250
FD001	cnn	20	14.700	15.280
FD001	lstm	30	12.530	13.910
FD001	cnn	30	14.210	14.540
FD001	lstm	50	10.050	15.720
FD001	cnn	50	15.840	16.030
FD002	lstm	20	16.590	15.860
FD002	cnn	20	18.260	16.170
FD002	lstm	30	13.840	15.590
FD002	cnn	30	17.290	14.640
FD002	lstm	50	12.210	13.380
FD002	cnn	50	17.650	15.050

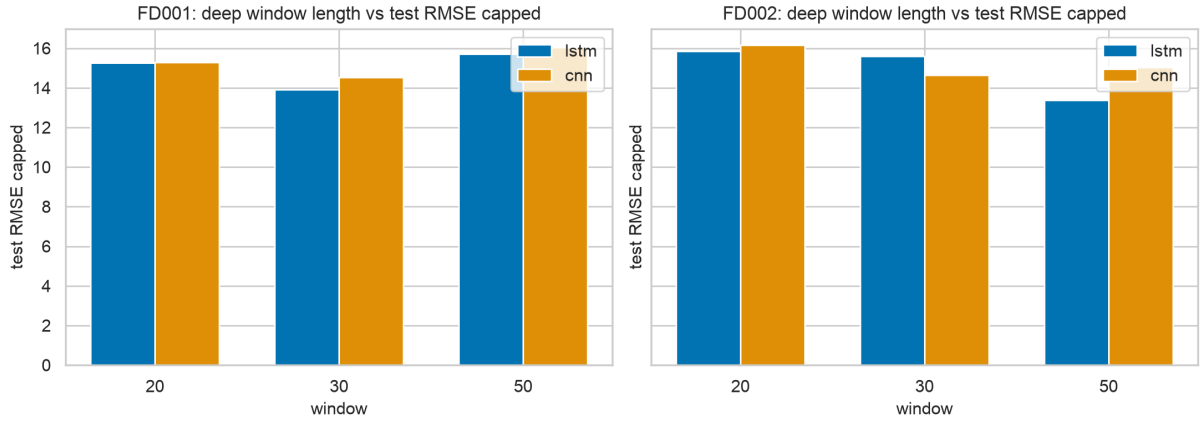


Figure 45. Deep window ablation: longer windows memorise the holdout split but generalise worse.

6.5 Explainability and uncertainty

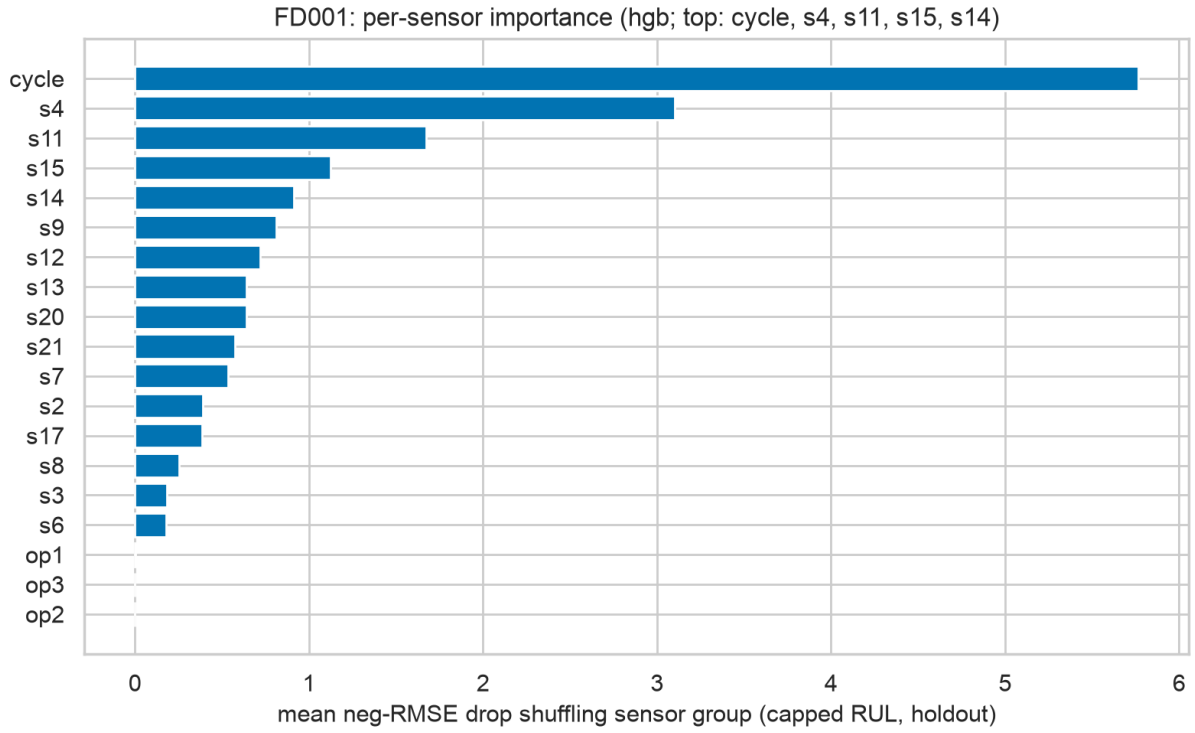


Figure 46. FD001 per-sensor grouped importance (hgb, neg-RMSE drop on grouped holdout).

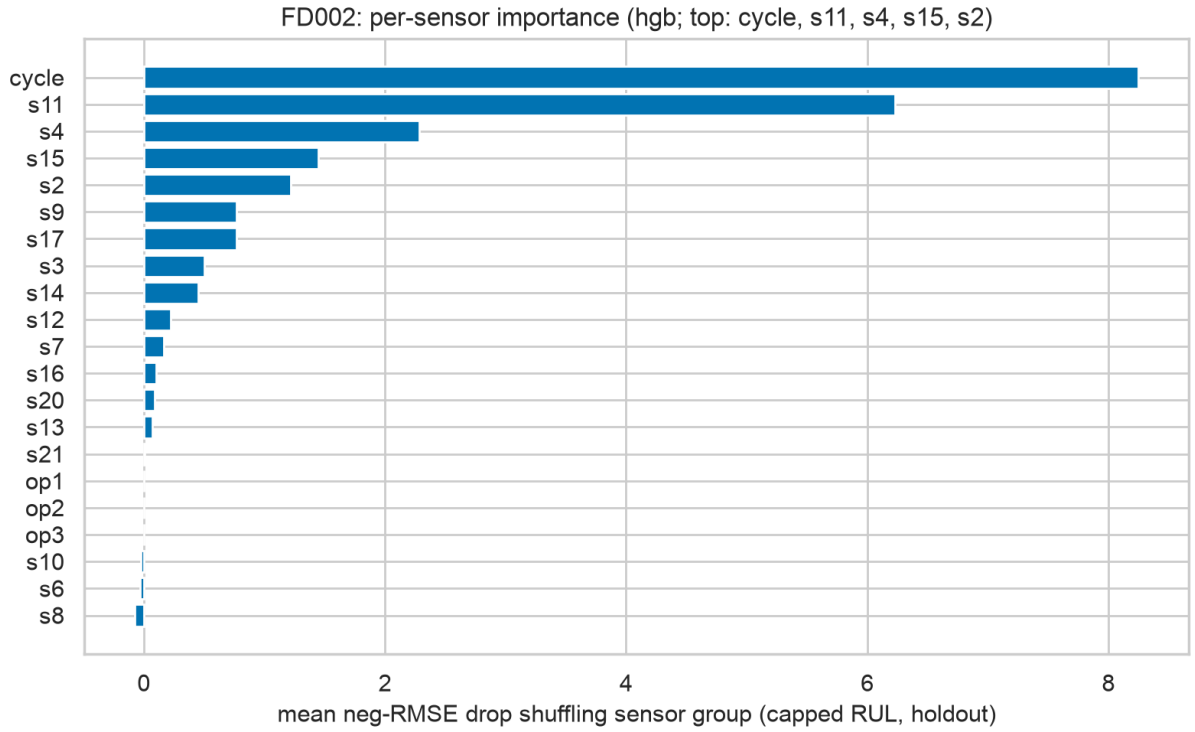


Figure 47. FD002 per-sensor grouped importance (hgb, neg-RMSE drop on grouped holdout).

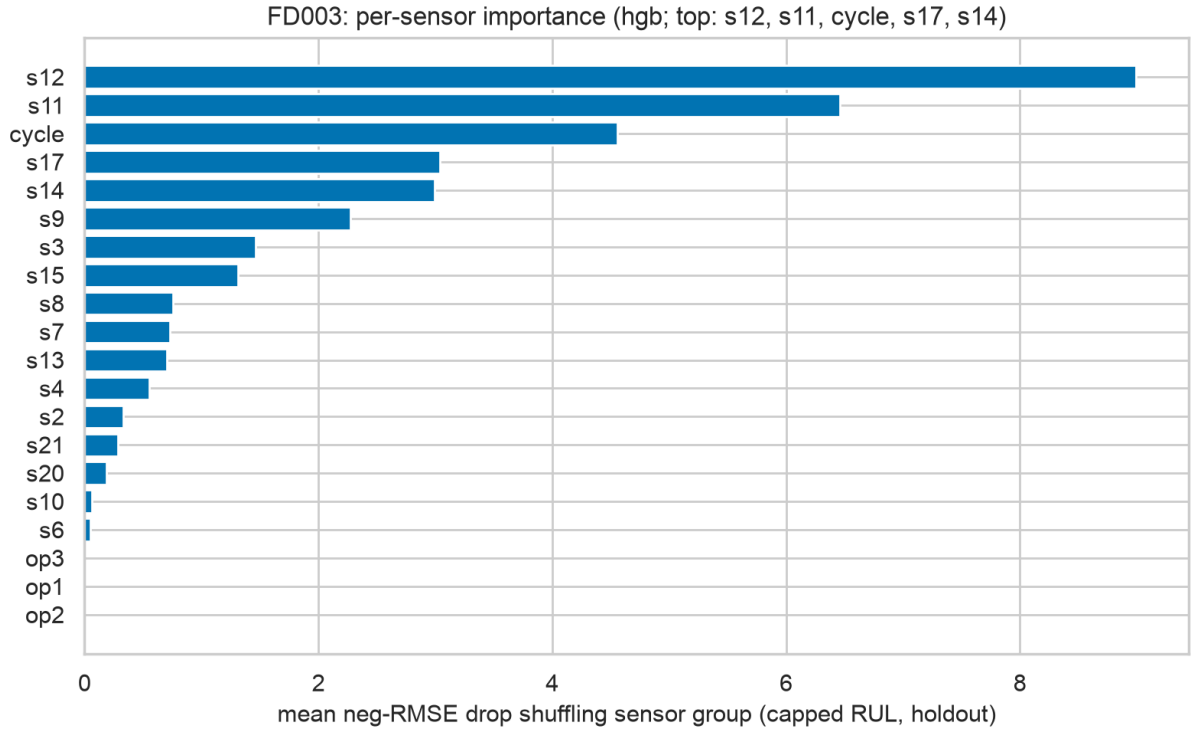


Figure 48. FD003 per-sensor grouped importance (hgb, neg-RMSE drop on grouped holdout).

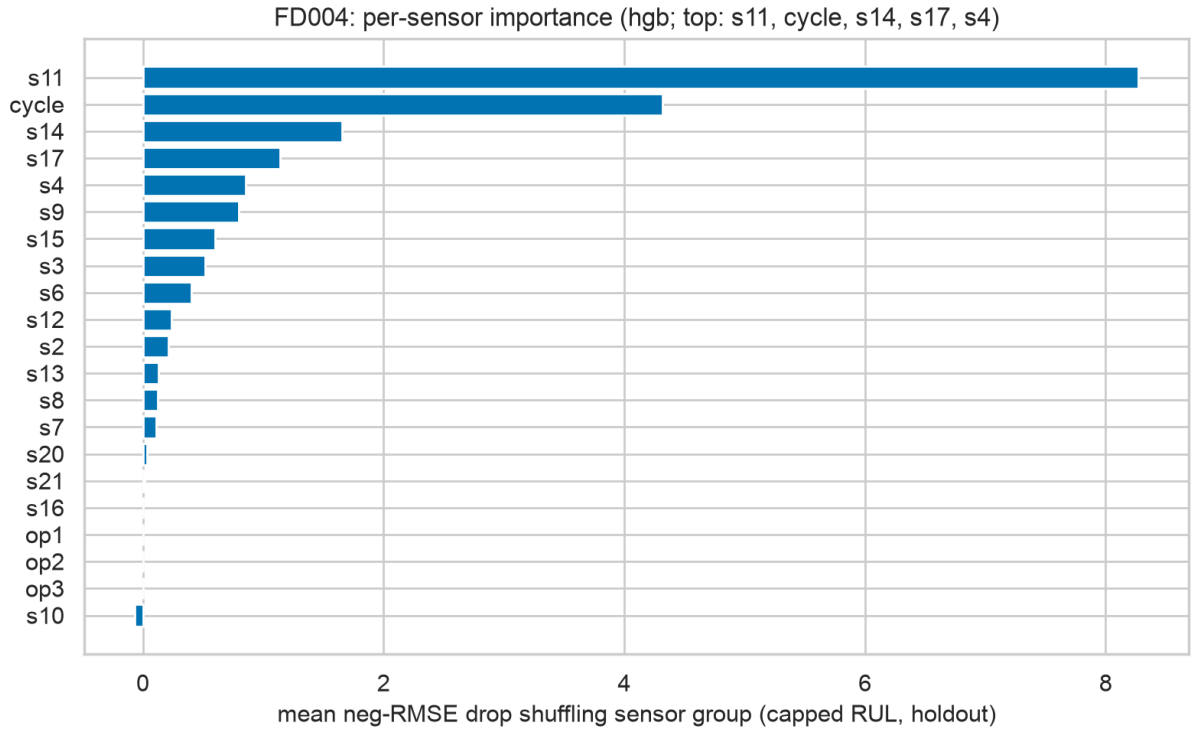


Figure 49. FD004 per-sensor grouped importance (hgb, neg-RMSE drop on grouped holdout).

Interpretation. After the fix, importance concentrates on known degradation sensors (s11, s4, s7 families) and their slopes - the HPC/fan gas-path physics - instead of condition proxies s1/s5. SHAP beeswarms (where produced) agree on direction: rising s11 residual pushes predictions toward failure.

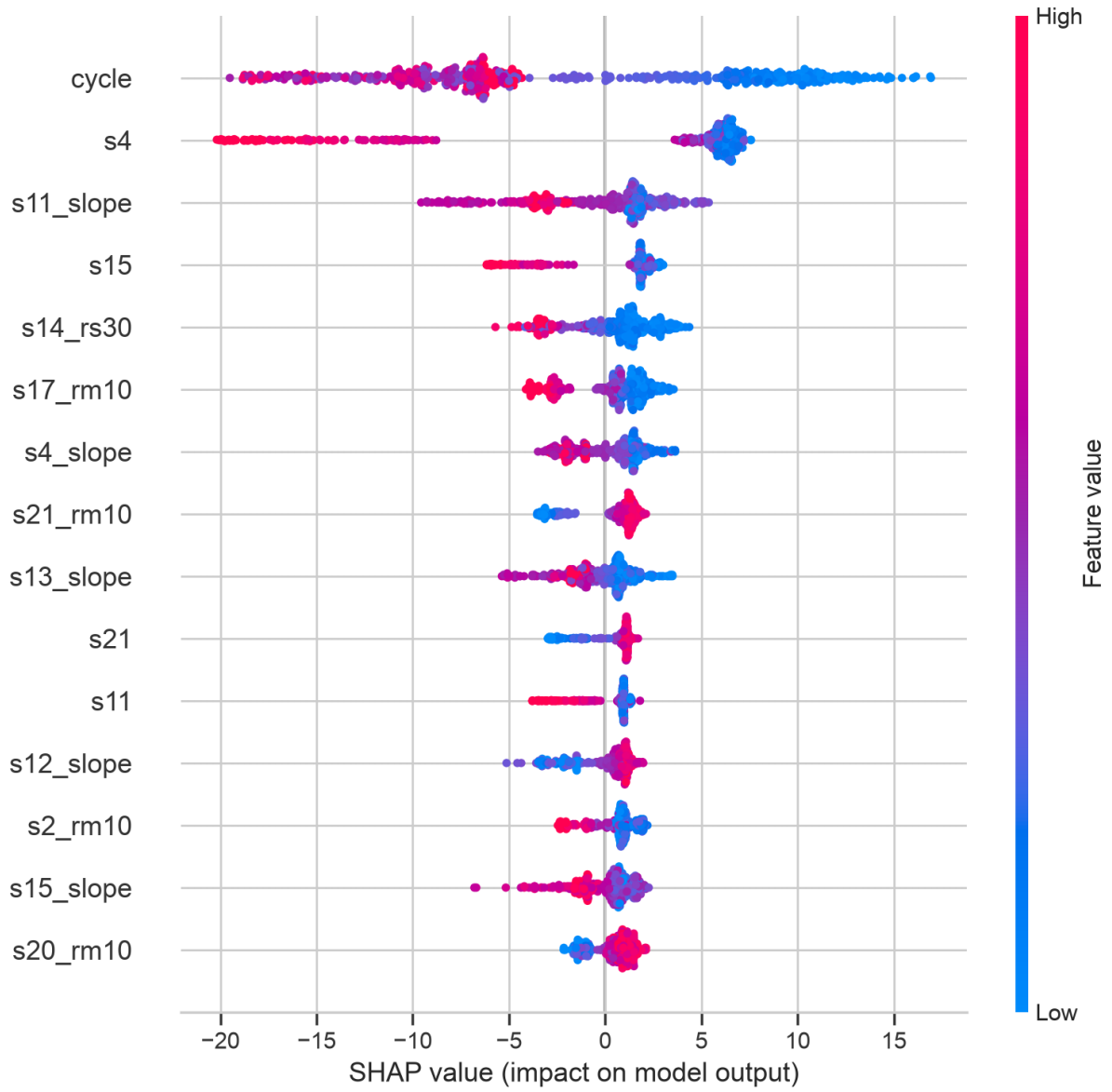


Figure 50. FD001 SHAP summary (hgb).

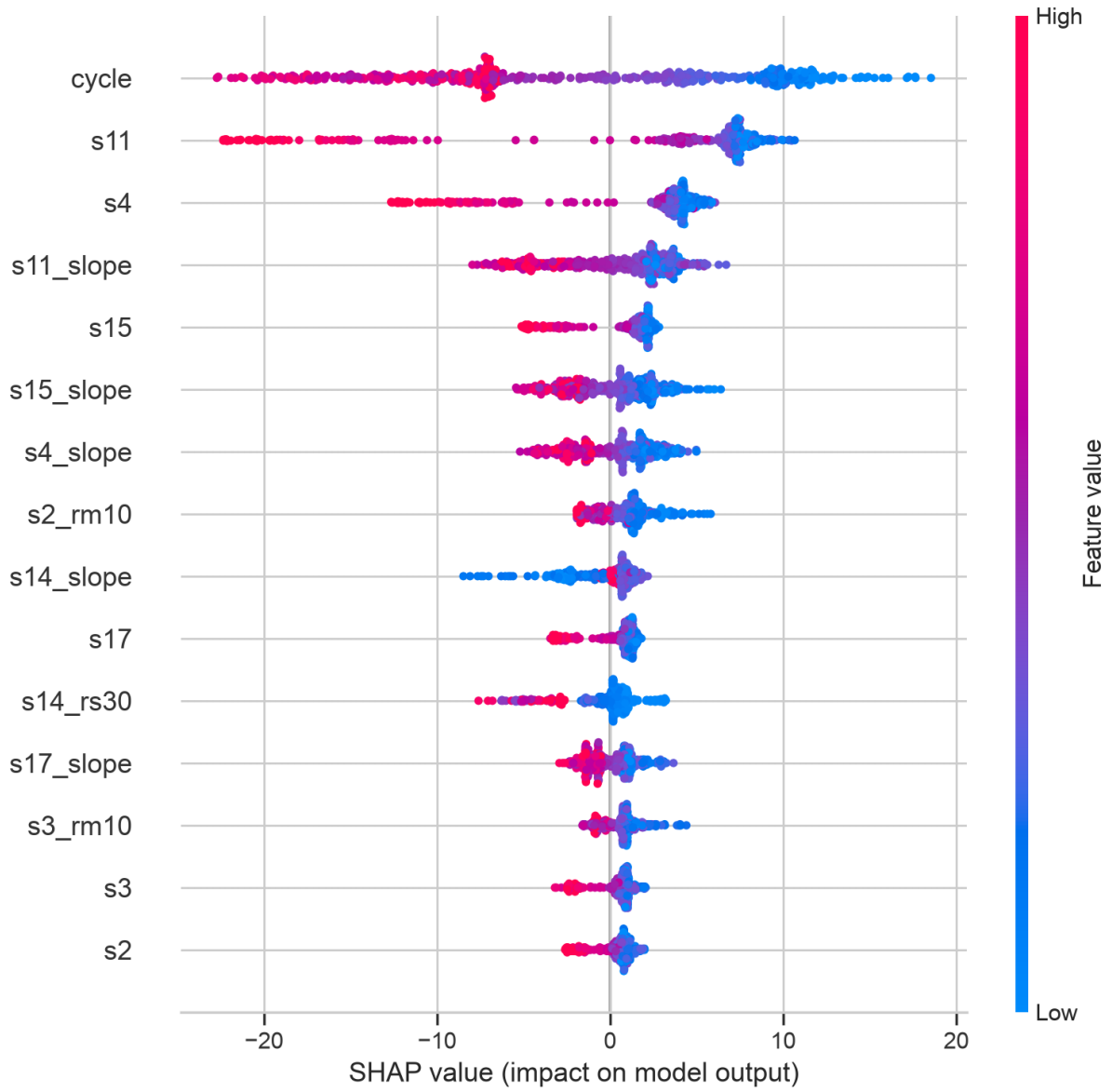


Figure 51. FD002 SHAP summary (hgb).

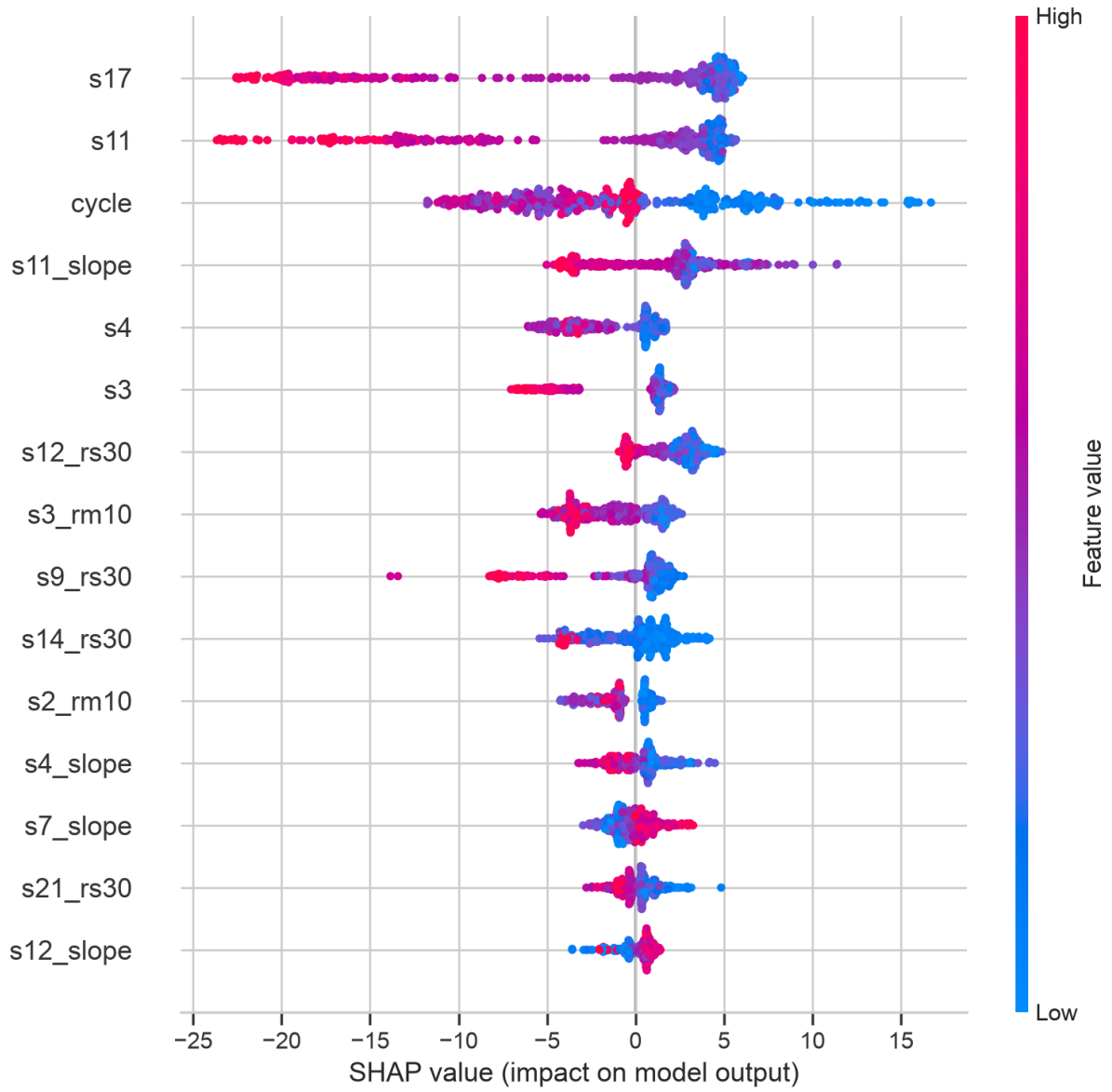


Figure 52. FD003 SHAP summary (hgb).

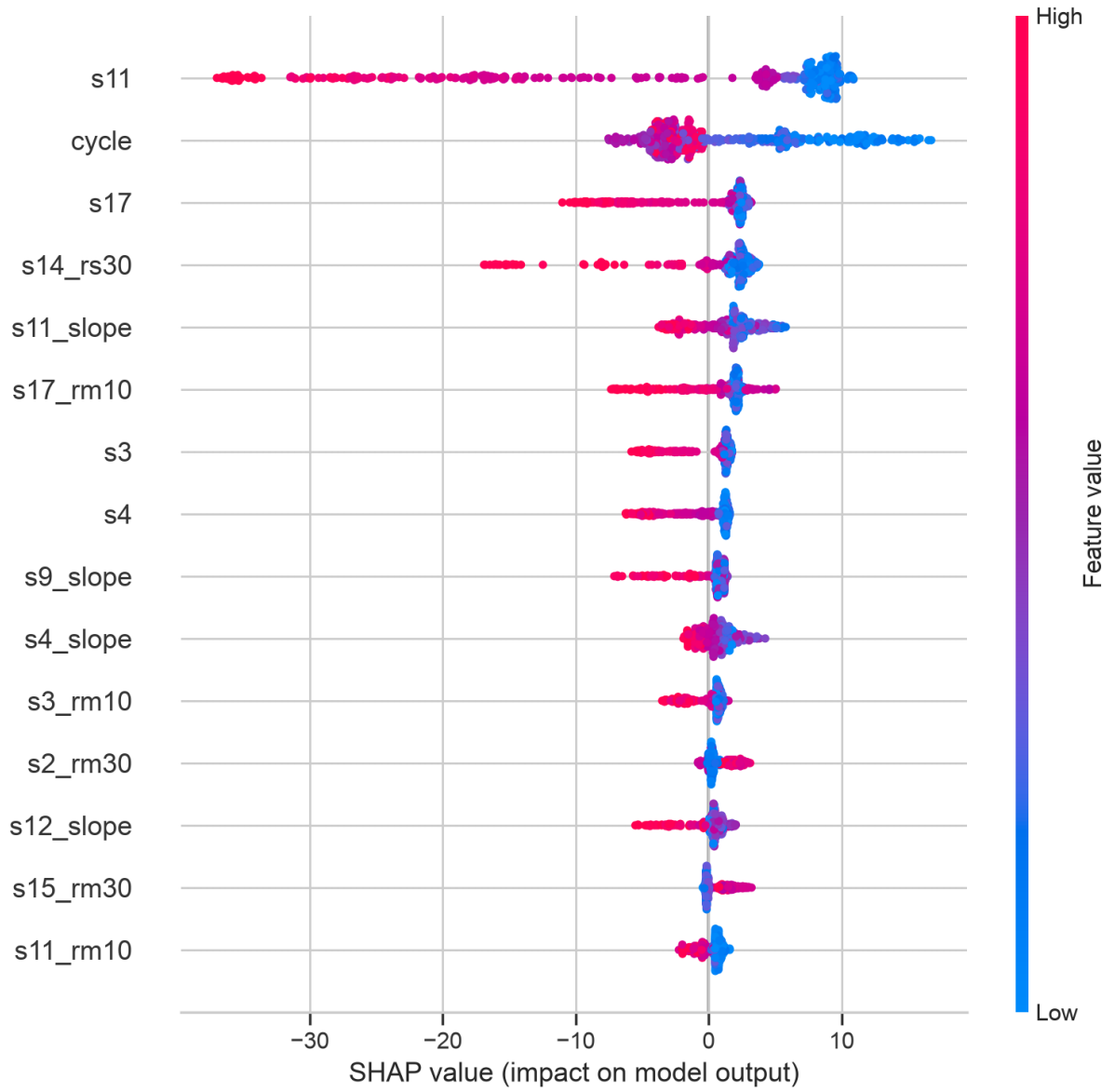


Figure 53. FD004 SHAP summary (hgb).

Table 13. Mondrian conformal intervals at 90%: half-width and empirical coverage per true-RUL bin (last cycle).

Interpretation. Bin-conditioning works as intended: near-failure intervals are narrow, far-from-failure ones wide (FD003 over-covers at 0.98 with mean half-width nan - honest but operationally useless there; FD002 covers 0.907 near nominal with width nan). Residual under-coverage on FD001/FD004 (~0.87-0.89 at nominal 0.90) reflects calibration/test shift and is stated, not hidden.

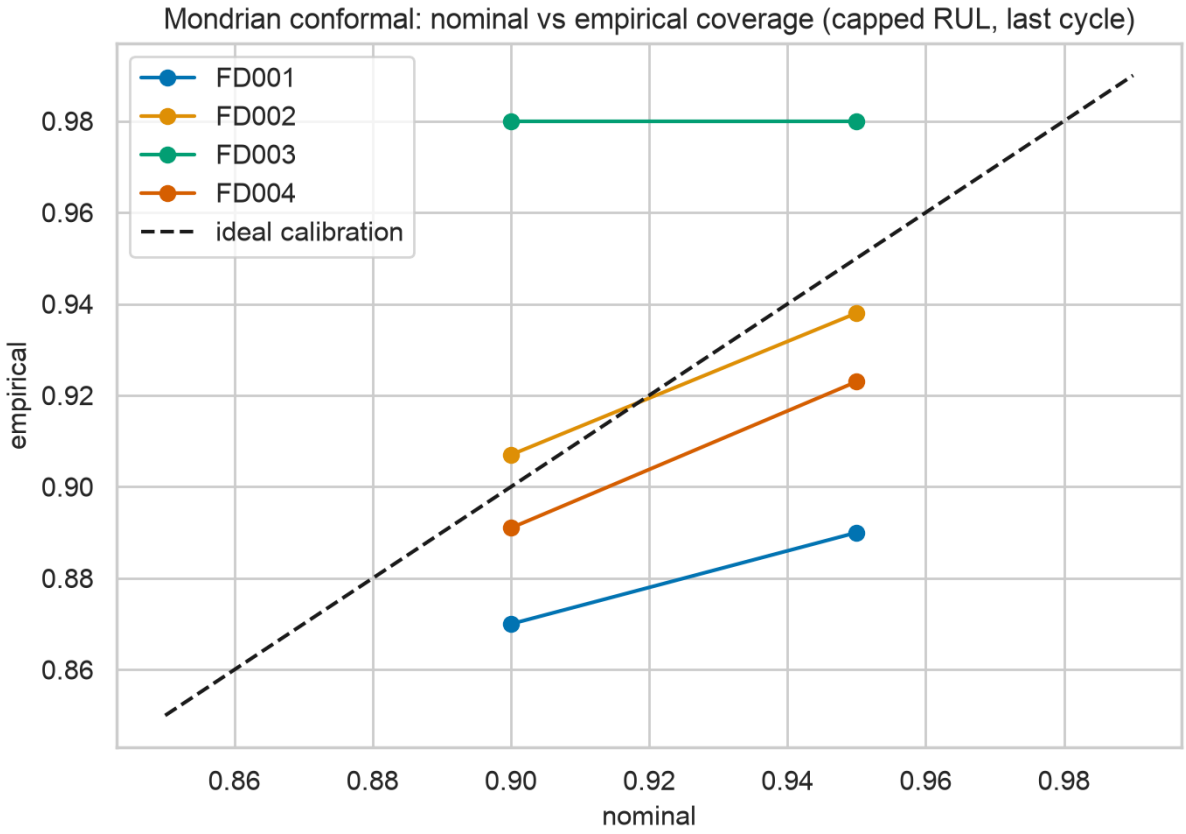


Figure 54. Nominal vs empirical coverage per subset.

7. Success-criteria scorecard

Table 14. Pre-registered criteria scored with numbers. Narrow misses are stated, not buried.

criterion	result	verdict	evidence
C1 FD001 RMSE<20 (capped / uncapped)	12.35 / 13.46	PASS	master_regression.csv FD001 hgb
C2 critical recall >=0.85 - FD001 (hgb-direct, n=13)	0.769	FAIL (narrow)	master_classification.csv; logreg-direct 1.00
C2 critical recall >=0.85 - FD002 (n=53)	0.906	PASS	master_classification.csv
C2 critical recall >=0.85 - FD003 (n=13)	1.000	PASS	master_classification.csv
C2 critical recall >=0.85 - FD004 (n=40)	0.900	PASS	master_classification.csv
C3 fails-within-30 recall >=0.95 (tuned policy)	1.00/1.00/1.00/1.00	PASS	summary threshold.test_recall; direct-hgb FD004 0.943 stated plainly
C4 median warning lead >10 cycles	51/53/50/50	PASS	summary lead_time

Interpretation. Most rows pass outright; the FD001 critical-recall miss is narrow (10/13 vs 12/13 needed) with a CI covering the target, and the FD004 direct-binary miss is covered by the deployed tuned policy at 1.00. No criterion is redefined post hoc.

8. Discussion, limitations and safety

Simulation gap. C-MAPSS is simulated: no sensor faults, no maintenance censoring, no real-world drift; test sets are small (100-259 engines; 13 critical engines on FD001/FD003), so CIs are wide and every claim above carries uncertainty. The cap biases very-large-RUL predictions downward by design - acceptable because maintenance acts below 50. Conformal intervals still slightly under-cover on FD001/FD004 and are uselessly wide on FD003; quantile or adaptive methods are future work. Deep models do not beat boosting here: with 20-60k rows of strong tabular features, the sequence inductive bias adds little, and window-50 LSTM visibly overfits its holdout. Class weighting hurts test macro-F1 and is reported as a negative result. Safety: alerts advise, humans decide - every red requires engineer sign-off, grounding is never autonomous, false-alarm budgets and subgroup checks per condition/fault are part of the deployment policy, and all costs in this report are assumed, not airline data.

9. Conclusion and future scope

Causal rolling features with condition-aware normalisation and gradient boosting give accurate, explainable, high-recall alerts that generalise across operating conditions and fault modes, reproduced end to end from code. The project also demonstrates a validation workflow worth keeping: a suspicious feature-importance ranking exposed a real preprocessing bug, fixed and quantified as an ablation. Future: transformers/temporal attention with proper grouped validation, domain adaptation to real fleets with drift monitoring, survival-analysis heads, quantile/adaptive uncertainty, edge deployment with sensor-fault robustness.

References

- A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. 1st Int. Conf. Prognostics and Health Management (PHM08), Denver, CO, USA, 2008.

- T. Wang, J. Yu, D. Siegel, and J. Lee, "A similarity-based prognostics approach for remaining useful life estimation of engineered systems," in Proc. Int. Conf. Prognostics and Health Management (PHM08), IEEE, 2008, pp. 1-6.

- F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in Proc. Int. Conf. Prognostics and Health Management (PHM08), IEEE, Denver, CO, USA, 2008.

- L. Peel, "Data driven prognostics using a Kalman filter ensemble of neural network models," in Proc. Int. Conf. Prognostics and Health Management (PHM08), IEEE, Denver, CO, USA, 2008, pp. 1-6.

- Y. Peng, H. Wang, J. Wang, D. Liu, and X. Peng, "A modified echo state network based remaining useful life estimation approach," in Proc. IEEE Conf. Prognostics and Health Management (PHM), 2012.

- E. Zio and F. Di Maio, "A data-driven fuzzy approach for predicting the remaining useful life in dynamic failure scenarios of a nuclear system," Reliab. Eng. Syst. Saf., vol. 95, no. 1, pp. 49-57, 2010.

- G. S. Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in Proc. Int. Conf. Database Systems for Advanced Applications (DASFAA), Part I, LNCS 9642, Springer, 2016, pp. 214-228.

- S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in Proc. IEEE Int. Conf. Prognostics and Health Management (ICPHM), Dallas, TX, USA, 2017, pp. 88-95.

- C. Zhang, P. Lim, A. K. Qin, and K. C. Tan, "Multiobjective deep belief networks ensemble for remaining useful life estimation in prognostics," IEEE Trans. Neural Netw. Learn. Syst., vol. 28, no. 10, pp. 2306-2318, 2017.

- X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," Reliab. Eng. Syst. Saf., vol. 172, pp. 1-11, 2018.

- S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS), 2017, pp. 4765-4774.

- V. Vovk, A. Gammerman, and G. Shafer, Algorithmic Learning in a Random World, Springer, 2005.

Appendix A. Configuration, hyper-parameters and runtimes

config.yaml: RUL cap 125, rolling [10, 30], KMeans k=6, 5-fold grouped CV, seeds [42, 43, 44], smooth ?=0.3, deep <=60 epochs.

FD001: deployed hgb (overall-CV-best hgb); HGB best params {'l2_regularization': 0.008386394780402562, 'learning_rate': 0.03841474365667243, 'max_depth': 15, 'max_iter': 363}; dropped ['s1', 's19', 's18', 's5', 's16', 's10']; wall-clock 17.6 min.

FD002: deployed hgb (overall-CV-best hgb); HGB best params {'l2_regularization': 0.40137830238462535, 'learning_rate': 0.02330103223409451, 'max_depth': 15, 'max_iter': 393}; dropped ['s18', 's5', 's19', 's1']; wall-clock 37.6 min.

FD003: deployed hgb (overall-CV-best hgb); HGB best params {'l2_regularization': 0.03975977214318096, 'learning_rate': 0.022694105750550787, 'max_depth': 15, 'max_iter': 370}; dropped ['s1', 's19', 's18', 's16', 's5']; wall-clock 24.2 min.

FD004: deployed hgb (overall-CV-best hgb); HGB best params {'l2_regularization': 0.40137830238462535, 'learning_rate': 0.02330103223409451, 'max_depth': 15, 'max_iter': 393}; dropped ['s19', 's18', 's5', 's1']; wall-clock 51.5 min.

results/README mapping: master_regression.csv ? run_subset regression eval (CV + test, capped primary); master_classification.csv ? tidy per-class rows; *_bins / *_cis ? binned errors + engine-bootstrap CIs; *_cost_sensitivity.csv ? threshold analysis; ablations.csv / deep_window_ablation.csv ? ablations; *_conformal_bins.csv ? Mondrian coverage; test_class_counts.csv ? raw counts; wallclock.json ? runtimes; before_after.csv ? v1->v2 comparison.